A BIT ABOUT DIGITAL

BILINGUAL COURSE BOOK · ENGLISH & KHMER · 12 SESSIONS

ការរចនាគេហទំព័រ ក្នុងយុគសម័យ AI

WEB DESIGN IN THE AI ERA

HTML · CSS · JAVASCRIPT

មូលដ្ឋានគ្រឹះ HTML, CSS និង JavaScript សម្រាប់សាកលវិទ្យាល័យ

How the Web Works | Semantic HTML | Design & UX | Flexbox & Grid | Responsive | DOM & APIs | Accessibility | AI-assisted Workflow

A BIT ABOUT DIGITAL · 2026

ការរចនាគេហទំព័រក្នុងយុគសម័យ AI

WEB DESIGN IN THE AI ERA

HTML, CSS & JAVASCRIPT IN 12 SESSIONS

A bilingual course book for a first university course in web design and front-end development. Each session has clear goals, explained examples that really run, a guided lab, and exercises. It also shows how to work with AI assistants responsibly.

សៀវភៅសិក្សាពីរភាសា សម្រាប់មុខវិជ្ជាដំបូងនៅសាកលវិទ្យាល័យ ផ្នែករចនាគេហទំព័រ និងការអភិវឌ្ឍផ្នែកខាងមុខ (front-end)។ វគ្គនីមួយៗមានគោលបំណងច្បាស់ ឧទាហរណ៍ដែលដំណើរការពិត លំហាត់អនុវត្តក្នុងបន្ទប់ពិសោធន៍ (lab) និងលំហាត់។ សៀវភៅនេះក៏បង្ហាញពីរបៀបធ្វើការជាមួយជំនួយការ AI ប្រកបដោយការទទួលខុសត្រូវផងដែរ។

A BIT ABOUT DIGITAL

Web Design in the AI Era: HTML, CSS & JavaScript in 12 Sessions
First edition, October 2026
Published by A Bit About Digital

This book may be shared and printed for non-commercial teaching and self-study.

How the examples were tested. Every listing in this book was saved as a file and run. HTML and CSS results are real screenshots taken in Chromium. Console results come from Node.js 22 or from Chromium. Fonts and form controls can look a little different in your browser and operating system.

Starter code. Each listing has a folder name such as `ex/s2_text`. The same files, plus a starter folder for every lab, come in the companion ZIP file.

About AI. The draft of this book was written with Claude, an AI model made by Anthropic, and every code example was run and checked. Treat any AI output, including this book, as something to test and verify.

អំពី AI។ សេចក្តីព្រាងនៃសៀវភៅនេះត្រូវបានសរសេរជាមួយ Claude ដែលជាម៉ូដែល AI របស់ Anthropic ហើយកូដឧទាហរណ៍ទាំងអស់ត្រូវបានដំណើរការ និងពិនិត្យ។ សូមចាត់ទុកលទ្ធផលពី AI ទាំងអស់ រួមទាំងសៀវភៅនេះ ជាអ្វីដែលត្រូវសាកល្បង និងធ្វើឯងផ្ទាត់។

PREFACE

អារម្ភកថា

Almost every business, school and government office in Cambodia now has a website or a web app. The three languages behind them have stayed the same for decades: **HTML** gives a page its structure, **CSS** gives it its look, and **JavaScript** makes it interactive. What has changed is how we work. AI assistants can now write a first draft of a page in seconds. That makes the fundamentals more important, not less: you need them to judge whether the draft is correct, accessible, secure and well designed.

> ស្ទើរតែគ្រប់អាជីវកម្ម សាលារៀន និងស្ថាប័នរដ្ឋនៅកម្ពុជា សុទ្ធតែមានគេហទំព័រ ឬកម្មវិធីវេប។ ភាសាទាំងបីដែលនៅពីក្រោយពួកវា នៅតែដដែលអស់រាប់ទសវត្សរ៍៖ **HTML** ផ្តល់រចនាសម្ព័ន្ធឱ្យទំព័រ **CSS** ផ្តល់រូបរាង ហើយ **JavaScript** ធ្វើឱ្យវាមានអន្តរកម្ម។ អ្វីដែលបានផ្លាស់ប្តូរគឺរបៀបដែលយើងធ្វើការ។ ជំនួយការ AI អាចសរសេរសេចក្តីព្រាងទំព័រមួយក្នុងរយៈពេលប៉ុន្មានវិនាទី។ ហេតុនេះមូលដ្ឋានគ្រឹះកាន់តែសំខាន់ មិនមែនតិចជាងមុនទេ៖ អ្នកត្រូវការវាដើម្បីវិនិច្ឆ័យថាសេចក្តីព្រាងនោះត្រឹមត្រូវ អាចប្រើបានសម្រាប់មនុស្សគ្រប់គ្នា មានសុវត្ថិភាព និងរចនាបានល្អឬអត់។

The book follows a running example, the small website of a fictional café called **Mekong Coffee**. In the labs, you build your own personal portfolio site step by step, and in Session 12 you publish it online.

> សៀវភៅនេះប្រើឧទាហរណ៍មួយតាំងពីដើមដល់ចប់ គឺគេហទំព័រតូចមួយរបស់ហាងកាហ្វេប្រឌិតឈ្មោះ **Mekong Coffee**។ នៅក្នុង lab អ្នកនឹងបង្កើតគេហទំព័រផលប័ត្រ (portfolio) ផ្ទាល់ខ្លួនមួយជំហានម្តងៗ ហើយនៅវគ្គទី ១២ អ្នកនឹងដាក់វាឱ្យដំណើរការលើអ៊ីនធឺណិត។

How to use this book

របៀបប្រើសៀវភៅនេះ

- `DEFINITION` boxes give the exact meaning of a term.
- `KEY IDEA` boxes hold rules worth remembering.
- `IN THE AI ERA` boxes show how to use AI assistants well for that topic, and what to check.
- Each **listing** shows the code, then the real result in the browser or console. Type the code yourself: your hands learn faster than your eyes.
- Each session ends with a **Lab** (about two hours) and **Exercises**. Hints and answers are in Appendix A.

- Numbers in square brackets, such as [1], point to the References.

> ប្រអប់ **និយមន័យ** ផ្តល់អត្ថន័យច្បាស់លាស់នៃពាក្យ។ ប្រអប់ **គំនិតសំខាន់** ផ្តុកច្បាប់ដែលគួរចងចាំ។ ប្រអប់ **ក្នុងយុគសម័យ AI** បង្ហាញពីរបៀបប្រើជំនួយការ AI ឱ្យបានល្អសម្រាប់ប្រធានបទនោះ និងអ្វីដែលត្រូវពិនិត្យ។ កូដនីមួយៗបង្ហាញលទ្ធផលពិតនៅក្នុង browser ឬ console។ វគ្គនីមួយៗបញ្ចប់ដោយ **Lab** (ប្រហែលពីរម៉ោង) និង **លំហាត់**។ ចម្លើយ និងការណែនាំមាននៅឧបសម្ព័ន្ធ A។

Course plan: 12 sessions

ផែនការមុខវិជ្ជា៖ ១២ វគ្គ

A suggested plan for one semester, with one session per week of about three hours: one hour of explanation and demo, then two hours of lab work.

#	Topic	Lab output
1	The web and web design in the AI era វេប និងការរចនាគេហទំព័រក្នុងយុគសម័យ AI	Tools installed, first page
2	HTML foundations: text, links, images, lists មូលដ្ឋាន HTML	Portfolio “About me” page
3	Semantic HTML, tables and forms HTML មានអត្ថន័យ តារាង និងទម្រង់	Structured pages + contact form
4	Design principles, UX and accessibility គោលការណ៍រចនា UX និងភាពងាយស្រួលប្រើ	Wireframe + style guide
5	CSS foundations: selectors, cascade, type, colour មូលដ្ឋាន CSS	Styled typography
6	Box model and Flexbox ម៉ូដែលប្រអប់ និង Flexbox	Header, navigation, cards
7	CSS Grid and responsive design CSS Grid និងការរចនាឆ្លើយតប	Mobile-first layout
8	Modern CSS: variables, states, dark mode CSS ទំនើប	Design tokens, dark theme
9	JavaScript foundations មូលដ្ឋាន JavaScript	Console exercises
10	The DOM and events DOM និងព្រឹត្តិការណ៍	Interactive to-do list
11	JSON, fetch and storing data JSON, fetch និងការរក្សាទុកទិន្នន័យ	Projects loaded from JSON
12	Quality, publishing and the AI-assisted workflow គុណភាព ការផ្សព្វផ្សាយ និងលំហូរការងារជាមួយ AI	Portfolio published online

Suggested assessment (adapt it to your faculty’s rules): weekly labs 30%, a mid-course mini project after Session 6 20%, the final portfolio project 40%, and participation 10%.

Appendix A includes a rubric for the final project.

> **ការវាយតម្លៃដែលបានស្នើ** (សូមកែសម្រួលតាមច្បាប់មហាវិទ្យាល័យរបស់អ្នក)៖ lab ប្រចាំសប្តាហ៍ 30% គម្រោងតូចបន្ទាប់ពីវគ្គទី ៦ 20% គម្រោងផលប័ត្រចុងក្រោយ 40% និងការចូលរួម 10%។ ឧបសម្ព័ន្ធ A មានតារាងវាយតម្លៃ (rubric) សម្រាប់គម្រោងចុងក្រោយ។

CONTENTS

មាតិកា

1 The Web and Web Design in the AI Era
1 វេប និងការរចនាគេហទំព័រក្នុងយុគសម័យ AI 11
1.1 How the web works 11
1.2 Three languages, three jobs 12
1.3 What is web design? 13
1.4 Your tools 13
1.5 Your first page 14
1.6 Working with AI assistants 15

2 HTML Foundations: Text, Links, Images and Lists
2 មូលដ្ឋាន HTML៖ អត្ថបទ តំណ រូបភាព និងបញ្ជី 18
2.1 Headings and paragraphs 18
2.2 Links 19
2.3 Images 20
2.4 Lists 21
2.5 Valid, clean HTML 22

3 Semantic HTML, Tables and Forms
3 HTML មានអត្ថន័យ តារាង និងទម្រង់ 25
3.1 Semantic elements 25
3.2 Tables for data 27
3.3 Forms 28

4 Design Principles, UX and Accessibility
4 គោលការណ៍រចនា UX និងភាពងាយស្រួលប្រើសម្រាប់មនុស្សគ្រប់គ្នា 32
4.1 A simple design process 32
4.2 Five visual principles 33
4.3 Usability heuristics 34
4.4 Accessibility and WCAG 2.2 35

5 CSS Foundations: Selectors, Cascade, Type and Colour
5 មូលដ្ឋាន CSS៖ Selector, Cascade, ពុម្ពអក្សរ និងពណ៌ 38
5.1 Adding CSS to a page 38
5.2 Selectors 39
5.3 The cascade and specificity 40
5.4 Units and colours 41
5.5 Styling text 41

6 **The Box Model and Flexbox**
ម៉ូដែលប្រអប់ និង Flexbox **45**
6.1 Every element is a box 45
6.2 Block, inline and inline-block 46
6.3 Flexbox: layout in one direction 47

7 **CSS Grid and Responsive Design**
CSS Grid និងការរចនាឆ្លើយតបតាមអេក្រង់ **51**
7.1 CSS Grid basics 51
7.2 Responsive, mobile-first design 52
7.3 Grids that adapt by themselves 53
7.4 Responsive images 55

8 **Modern CSS: Variables, States and Dark Mode**
CSS ទំនើប៖ អថេរ ស្ថានភាព និងរបៀបងងឹត **57**
8.1 Custom properties as design tokens 57
8.2 Respecting user settings 58
8.3 Interactive states and transitions 59
8.4 Thinking in components 60

9 **JavaScript Foundations**
មូលដ្ឋាន JavaScript **62**
9.1 Where JavaScript runs 62
9.2 Values, variables and types 63
9.3 Functions, decisions and loops 64
9.4 Arrays and objects 65

10 **The DOM and Events**
DOM និងព្រឹត្តិការណ៍ **68**
10.1 What is the DOM? 68
10.2 Events 70
10.3 Forms and creating elements 72

11 **JSON, Fetch and Storing Data**
JSON, Fetch និងការរក្សាទុកទិន្នន័យ **75**
11.1 JSON 75
11.2 Fetching data 76
11.3 Saving data in the browser 78
11.4 APIs and secrets 79

12 **Quality, Publishing and the AI-assisted Workflow**
គុណភាព ការផ្សព្វផ្សាយ និងលំហូរការងារជាមួយ AI **81**
12.1 An accessibility check 81
12.2 Lighthouse, performance and SEO 83
12.3 Version control with Git 83

12.4 Publishing with GitHub Pages 84
12.5 A responsible AI-assisted workflow 84

A **Answers, Hints and Project Rubric**
ចម្លើយ ការណែនាំ និងតារាងវាយតម្លៃ **87**

B **Quick Reference**
ឯកសារយោងរហ័ស **91**

C **Glossary**
សទ្ទានុក្រម **92**

References
ឯកសារយោង **93**

SESSION 1

THE WEB AND WEB DESIGN IN THE AI ERA

វេប និងការរចនាគេហទំព័រក្នុងយុគសម័យ AI

IN THIS SESSION ក្នុងវគ្គនេះ

- Explain what happens between typing a web address and seeing a page
- Name the job of HTML, CSS and JavaScript
- Set up a code editor and the browser developer tools
- Write and open your first HTML page
- Agree on responsible rules for using AI in this course

- ពន្យល់ពីអ្វីដែលកើតឡើងចាប់ពីការវាយអាសយដ្ឋានវេបរហូតដល់ឃើញទំព័រ
- ប្រាប់តួនាទីរបស់ HTML, CSS និង JavaScript
- ដំឡើងកម្មវិធីសរសេរកូដ និងឧបករណ៍អ្នកអភិវឌ្ឍក្នុង browser
- សរសេរ និងបើកទំព័រ HTML ដំបូងរបស់អ្នក
- ព្រមព្រៀងលើច្បាប់ប្រើ AI ប្រកបដោយការទទួលខុសត្រូវ

1.1 How the web works

រៀបដែលវេបដំណើរការ

The web is a system of documents and apps that computers request from each other over the internet. Two kinds of computers take part. A **client** is your device and its web browser. A **server** is a computer that stores websites and sends copies of their files when a client asks [1].

When you type `https://www.example.com` and press Enter, four things happen. The browser asks the **DNS** (Domain Name System), which works like an address book, for the real IP address of the server. It then sends an **HTTP request** to that server. The server answers with an **HTTP response** that has a status code, such as `200 OK` when everything worked or `404 Not Found` when the file does not exist [2]. Finally, the browser reads the HTML, downloads the CSS, images and JavaScript it mentions, and draws the page.

1 · TYPE
You enter a URL in the browser.
វាយ URL ក្នុង browser

2 · DNS
The name is turned into an IP address.
ប្តូរឈ្មោះទៅជាអាសយដ្ឋាន IP

3 · REQUEST
The browser sends an HTTP request.
browser ផ្ញើសំណើ HTTP

4 · RESPONSE
The server replies; the browser draws the page.
server ឆ្លើយតប ហើយ browser បង្ហាញទំព័រ

> វេប (web) គឺជាប្រព័ន្ធឯកសារ និងកម្មវិធីដែលកុំព្យូទ័រស្នើសុំពីគ្នាទៅវិញទៅមកតាមអ៊ីនធឺណិត។ **Client** គឺជាឧបករណ៍ និង browser របស់អ្នក។ **Server** គឺជាកុំព្យូទ័រដែលរក្សាទុកគេហទំព័រ ហើយផ្ញើច្បាប់ចម្លងនៃឯកសារពេល client ស្នើសុំ។ ពេលអ្នកវាយអាសយដ្ឋានមួយ browser សួរ **DNS** (ដូចសៀវភៅអាសយដ្ឋាន) ដើម្បីរកអាសយដ្ឋាន IP ពិតរបស់ server បន្ទាប់មកផ្ញើ **សំណើ HTTP**។ Server ឆ្លើយតបដោយ **ចម្លើយ HTTP** ដែលមានលេខកូដស្ថានភាព ដូចជា `200 OK` ពេលដំណើរការល្អ ឬ `404 Not Found` ពេលរកឯកសារមិនឃើញ។ ចុងក្រោយ browser អាន HTML ទាញយក CSS រូបភាព និង JavaScript ហើយគូរទំព័រ។

Anatomy of a URL · សមាសធាតុនៃ URL

Part	Example	Meaning	អត្ថន័យ
Scheme	`https://`	Protocol; the S means encrypted	ពិធីការ (S = មានការអ៊ិនគ្រីប)
Host	`www.example.com`	Domain name of the server	ឈ្មោះដែនរបស់ server
Path	`/menu/drinks.html`	Which file or page	ឯកសារ ឬទំព័រណា
Query	`?size=large`	Extra data for the server	ទិន្នន័យបន្ថែមសម្រាប់ server
Fragment	`#prices`	A place inside the page	ទីតាំងមួយក្នុងទំព័រ

1.2 Three languages, three jobs

ភាសាបី តួនាទីបី

Think of a website as a house. **HTML** (HyperText Markup Language) is the structure: walls, rooms and doors. It says “this is a heading”, “this is a list”, “this is a link”. **CSS** (Cascading Style Sheets) is the decoration: colours, fonts, spacing and layout. **JavaScript** is the electricity: it makes things react when the user clicks, types or scrolls. Keeping the three separate makes a site easier to change, to reuse and to fix.

> សូមគិតពីគេហទំព័រដូចផ្ទះមួយ។ **HTML** គឺជារចនាសម្ព័ន្ធ៖ ជញ្ជាំង បន្ទប់ និងទ្វារ។ វាប្រាប់ថា «នេះជាចំណងជើង» «នេះជាបញ្ជី» «នេះជាតំណ»។ **CSS** គឺជាការតុបតែង៖ ពណ៌ ពុម្ពអក្សរ គម្លាត និងប្លង់។ **JavaScript** គឺជាអគ្គិសនី៖ វាធ្វើឱ្យទំព័រមានប្រតិកម្មពេលអ្នកប្រើចុច វាយ ឬរំកិល។ ការបំបែកភាសាទាំងបីធ្វើឱ្យងាយស្រួលកែប្រែប្រើឡើងវិញ និងជួសជុល។

DEFINITION ន័យមន័យ

Front-end code runs in the user's browser (HTML, CSS, JavaScript). **Back-end** code runs on the server and handles things like databases, logins and payments. This course is about the front end.

កូដ **front-end** ដំណើរការក្នុង browser របស់អ្នកប្រើ (HTML, CSS, JavaScript)។ កូដ **back-end** ដំណើរការលើ server ហើយគ្រប់គ្រងមូលដ្ឋានទិន្នន័យ ការចូលគណនី និងការទូទាត់។ មុខវិជ្ជានេះផ្តោតលើ front-end។

1.3 What is web design?

តើការរចនាគេហទំព័រជាអ្វី?

Web design is more than making a page pretty. A well-designed site helps real people reach a goal quickly: find opening hours, buy a ticket, apply for a course. Designers think about **content** (what to say), **user experience** or UX (how easy the journey is), **user interface** or UI (how each screen looks and behaves), **accessibility** (whether people with disabilities can use it) and **performance** (how fast it loads on a cheap phone and a slow network). Session 4 covers these principles in detail.

ការរចនាគេហទំព័រមិនមែនគ្រាន់តែធ្វើឱ្យទំព័រស្អាតទេ។ គេហទំព័រដែលរចនាល្អជួយមនុស្សពិតឱ្យសម្រេចគោលដៅបានលឿន៖ រកម៉ោងបើក ទិញសំបុត្រ ឬដាក់ពាក្យចូលរៀន។ អ្នករចនាគិតពី **មាតិកា បទពិសោធន៍អ្នកប្រើ (UX) ផ្ទៃប្រើប្រាស់ (UI) ភាពងាយស្រួលប្រើសម្រាប់មនុស្សគ្រប់គ្នា (accessibility)** និង **ល្បឿន (performance)** ជាពិសេសលើទូរស័ព្ទតម្លៃថោក និងបណ្តាញយឺត។

1.4 Your tools

ឧបករណ៍របស់អ្នក

You need only two free tools. A **code editor** such as Visual Studio Code helps you write code with colours, auto-complete and error hints. A modern **browser** such as Chrome, Firefox or Edge shows your page and includes **developer tools** (DevTools). Press `F12` (or right-click and choose *Inspect*) to open them. The *Elements* panel shows the HTML and CSS of any page, the *Console* panel shows JavaScript messages and errors, and the *Network* panel shows every request the page makes.

Organise your work from day one. Keep each project in its own folder. Name the home page `index.html`. Use lowercase file names without spaces, for example `about-me.html`, because web servers often treat `About.html` and `about.html` as different files.

អ្នកត្រូវការតែឧបករណ៍ឥតគិតថ្លៃពីរ។ **កម្មវិធីសរសេរកូដ** ដូចជា Visual Studio Code ជួយអ្នកសរសេរកូដដោយមានពណ៌ ការបំពេញស្វ័យប្រវត្តិ និងការព្រមានកំហុស។ **Browser** ទំនើបដូចជា Chrome, Firefox ឬ

Edge បង្ហាញទំព័ររបស់អ្នក និងមាន **ឧបករណ៍អ្នកអភិវឌ្ឍ (DevTools)**។ ចុច `F12` ដើម្បីបើកវា។ ផ្ទាំង *Elements* បង្ហាញ HTML និង CSS ផ្ទាំង *Console* បង្ហាញសារ និងកំហុស JavaScript ហើយផ្ទាំង *Network* បង្ហាញសំណើទាំងអស់។ ទុកគម្រោងនីមួយៗក្នុងថតរៀងៗខ្លួន ដាក់ឈ្មោះទំព័រដើមថា `index.html` ហើយប្រើឈ្មោះឯកសារជាអក្សរតូច គ្មានដកឃ្លា។

1.5 Your first page

ទំព័រដំបូងរបស់អ្នក

Create a file called `index.html`, type Listing 1.1, save it and open it in your browser by double-clicking the file.

HTML INDEX.HTML LISTING 1.1 · EX/S1_FIRST

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>My First Page</title>
</head>
<body>
  <h1>Hello, Phnom Penh!</h1>
  <p>This is my first web page. I wrote it by hand.</p>
</body>
</html>
```

RESULT IN THE BROWSER · លទ្ធផល CHROMIUM

Hello, Phnom Penh!

This is my first web page. I wrote it by hand.

560px wide

Every line has a reason. `<!DOCTYPE html>` tells the browser to use modern standards mode. The `<html lang="en">` element wraps the whole page, and its `lang` attribute tells screen readers and translation tools the language; use `lang="km"` for a Khmer page. The `<head>` holds information about the page: `charset="UTF-8"` lets the page show Khmer and every other script correctly, the `viewport` line makes the page fit phone screens, and `<title>` is the text in the browser tab and in search results. Everything the user sees goes inside `<body>` [3].

បន្ទាត់នីមួយៗមានហេតុផល។ `<!DOCTYPE html>` ប្រាប់ browser ឱ្យប្រើស្តង់ដារទំនើប។ ធាតុ `<html lang="en">` រុំទំព័រទាំងមូល ហើយ `lang` ប្រាប់កម្មវិធីអានអេក្រង់ពីភាសា (ប្រើ `lang="km"` សម្រាប់ទំព័រខ្មែរ)។ `<head>` ផ្ទុកព័ត៌មានអំពីទំព័រ៖ `charset="UTF-8"` ធ្វើឱ្យអក្សរខ្មែរបង្ហាញត្រឹមត្រូវ បន្ទាត់ `viewport`

ធ្វើឱ្យទំព័រសមនឹងអេក្រង់ទូរស័ព្ទ ហើយ `<title>` ជាអត្ថបទលើផ្ទាំង browser។ អ្វីៗដែលអ្នកប្រើមើលឃើញដាក់ក្នុង `<body>` ។

KEY IDEA គំនិតសំខាន់

An HTML **element** is usually a start tag, content and an end tag: `<p>Hello</p>`. **Attributes** add extra information inside the start tag: `<html lang="en">`. Elements can sit inside other elements, like boxes inside boxes, but they must close in the reverse order they opened.

ធាតុ (element) HTML ជាធម្មតាមានស្លាកបើក មាតិកា និងស្លាកបិទ។ **គុណលក្ខណៈ (attribute)** បន្ថែមព័ត៌មានក្នុងស្លាកបើក។ ធាតុអាចនៅក្នុងធាតុផ្សេងទៀតដូចប្រអប់ក្នុងប្រអប់ ប៉ុន្តែត្រូវបិទតាមលំដាប់បញ្ច្រាសនៃការបើក។

1.6 Working with AI assistants

ការធ្វើការជាមួយជំនួយការ AI

AI assistants such as Claude, ChatGPT and Gemini can explain code, suggest fixes, draft a layout or turn a sketch into HTML. Used well, they are like a patient tutor who is available at midnight. Used badly, they hide gaps in your understanding until an exam, an interview or a real client exposes them. AI output can also be wrong in ways that look right: outdated code, invented attributes, missing accessibility features or security mistakes. You are responsible for every line you submit or publish.

ជំនួយការ AI ដូចជា Claude, ChatGPT និង Gemini អាចពន្យល់កូដ ស្នើការកែកំហុស ព្រាងប្លង់ ឬបំប្លែងគំនូរព្រាងទៅជា HTML។ បើប្រើបានល្អ វាដូចគ្រូបង្រៀនដែលអត់ធ្មត់ និងនៅជាមួយអ្នកពេលពាក់កណ្តាលអធ្រាត្រ។ បើប្រើមិនល្អ វាលាក់ចន្លោះខ្វះខាតនៃការយល់ដឹងរបស់អ្នក រហូតដល់ការប្រឡង ការសម្ភាសការងារ ឬអតិថិជនពិតបង្ហាញវាចេញ។ លទ្ធផល AI ក៏អាចខុស ទោះមើលទៅដូចត្រូវ៖ កូដហួសសម័យ គុណលក្ខណៈប្រឌិត ខ្វះភាពងាយស្រួលប្រើ ឬកំហុសសុវត្ថិភាព។ អ្នកទទួលខុសត្រូវលើគ្រប់បន្ទាត់ដែលអ្នកដាក់ជូន ឬផ្សព្វផ្សាយ។

IN THE AI ERA ក្នុងយុគសម័យ AI

Five rules for this course.

- **Understand first.** Do not submit code you cannot explain line by line.
- **Ask to learn, not only to finish.** "Explain why my image does not show" teaches more than "fix my page".
- **Verify.** Check answers against MDN Web Docs [4] and test them in the browser.
- **Protect privacy.** Never paste passwords, API keys or other people's personal data into an AI tool.
- **Be honest.** Follow your lecturer's policy and say when and how you used AI.

ច្បាប់ប្រាំ៖ យល់ជាមុនសិន (កុំដាក់ជូនកូដដែលអ្នកពន្យល់មិនបាន) សួរដើម្បីរៀន មិនមែនគ្រាន់តែឱ្យចប់ ធ្វៀងផ្ងាត់ជាមួយ MDN និងសាកល្បងក្នុង browser ការពារឯកជនភាព (កុំបិទភ្ជាប់ពាក្យសម្ងាត់ ឬទិន្នន័យផ្ទាល់ខ្លួនអ្នកដទៃ) និងស្មោះត្រង់ (ប្រាប់ពេល និងរបៀបដែលអ្នកប្រើ AI)។

A good prompt gives context and constraints, just as you would brief a classmate [5]. Compare "make a website" with "I am a first-year student. Explain, line by line, what each tag in my `index.html` does. Do not rewrite it." The second prompt makes the assistant teach you.

> ប្រអប់បញ្ជូល (prompt) ល្អផ្តល់បរិបទ និងលក្ខខណ្ឌ ដូចពេលអ្នកពន្យល់ទៅមិត្តរួមថ្នាក់។ ប្រៀបធៀប «បង្កើតគេហទំព័រមួយ» ជាមួយ «ខ្ញុំជានិស្សិតឆ្នាំទីមួយ។ សូមពន្យល់ម្តងមួយបន្ទាត់ថាស្លាកនីមួយៗក្នុង index.html របស់ខ្ញុំធ្វើអ្វី។ កុំសរសេរវាឡើងវិញ។» prompt ទីពីរធ្វើឱ្យជំនួយការបង្រៀនអ្នក។

LAB 1 · SET UP YOUR WORKSPACE Lab ១ · រៀបចំឧបករណ៍

1. Install a code editor and two browsers. Create a folder `portfolio`.
   ដំឡើងកម្មវិធីសរសេរកូដ និង browser ពីរ។ បង្កើតថត portfolio។
2. Type Listing 1.1 into `portfolio/index.html`. Change the heading to your name and the paragraph to one sentence about you.
   វាយ Listing 1.1 ហើយប្តូរចំណងជើងទៅជាឈ្មោះរបស់អ្នក។
3. Open the page. Press F12, find your heading in *Elements* and change its text in DevTools. Reload: what happens, and why?
   បើក DevTools ប្តូរអត្ថបទ ហើយ reload។ តើមានអ្វីកើតឡើង? ហេតុអ្វី?
4. Open the *Network* panel and reload any news website. Find one request with status 200 and, if you can, one with another status.
   ក្នុងផ្ទាំង Network រកសំណើដែលមានស្ថានភាព 200។

5. Ask an AI assistant to explain your page line by line. Write down one thing it explained well and one thing you had to check on MDN.
   សុំ AI ពន្យល់ទំព័ររបស់អ្នក ហើយកត់ត្រាអ្វីដែលអ្នកត្រូវពិនិត្យលើ MDN។

Exercises

លំហាត់

1. In your own words, explain the difference between a client and a server.
   ពន្យល់ពីភាពខុសគ្នារវាង client និង server។
2. Split `https://shop.example.com/products/tea.html?color=green#reviews` into scheme, host, path, query and fragment.
   បំបែក URL នេះជាផ្នែកៗ។
3. What does status code 404 mean? Who sends it?
   តើលេខកូដ 404 មានន័យដូចម្តេច? នរណាផ្ញើវា?
4. Give one example each of a task for HTML, CSS and JavaScript on a café website.
   ផ្តល់ឧទាហរណ៍មួយសម្រាប់ HTML, CSS និង JavaScript។
5. Write two prompts for an AI assistant about Listing 1.1: one that would make you learn, and one that would not. Explain the difference.
   សរសេរ prompt ពីរ៖ មួយជួយអ្នករៀន មួយទៀតមិនជួយ។

SESSION 2

HTML FOUNDATIONS: TEXT, LINKS, IMAGES AND LISTS

មូលដ្ឋាន HTML៖ អត្ថបទ តំណ រូបភាព និងបញ្ជី

IN THIS SESSION ក្នុងវគ្គនេះ

- Mark up headings and paragraphs with the right meaning
- Link pages together with relative and absolute URLs
- Add images with useful alternative text
- Build ordered, unordered and nested lists
- Check your HTML with a validator

- សរសេរចំណងជើង និងកថាខណ្ឌឱ្យមានអត្ថន័យត្រឹមត្រូវ
- ភ្ជាប់ទំព័រដោយ URL ទាក់ទង និង URL ពេញ
- បន្ថែមរូបភាពជាមួយអត្ថបទជំនួស (alt) ដែលមានប្រយោជន៍
- បង្កើតបញ្ជីមានលេខ គ្មានលេខ និងបញ្ជីក្នុងបញ្ជី
- ពិនិត្យ HTML ជាមួយ validator

2.1 Headings and paragraphs

ចំណងជើង និងកថាខណ្ឌ

HTML has six levels of headings, `<h1>` to `<h6>`. Use them like the outline of a report: one `<h1>` for the page topic, `<h2>` for main sections, `<h3>` for sub-sections. Do not pick a heading because of its size; size is the job of CSS. Screen-reader users often jump from heading to heading, so a clear outline is also an accessibility feature.

Text goes in paragraphs, `<p>`. Inside a paragraph, `<strong>` marks strong importance and `<em>` marks stress emphasis. The browser shows them in bold and italic, but their real value is meaning. `<br>` forces a line break and is fine for addresses or poems, not for spacing. `<hr>` marks a change of topic. Special characters are written as **entities**, for example `©` for ©, `<` for < and `&` for &.

HTML INDEX.HTML LISTING 2.1 · EX/S2_TEXT

```html
<h1>Mekong Coffee</h1>
<h2>About us</h2>
<p>We roast <strong>Cambodian coffee</strong> in small batches.
   Our beans come from <em>Mondulkiri</em> farms.</p>
<h2>Opening hours</h2>
<p>Monday to Saturday<br>7:00 to 18:00</p>
<hr>
<p><small>&copy; 2026 Mekong Coffee</small></p>
```

RESULT IN THE BROWSER · លទ្ធផល CHROMIUM

Mekong Coffee

About us

We roast **Cambodian coffee** in small batches. Our beans come from *Mondulkiri* farms.

Opening hours

Monday to Saturday
7:00 to 18:00

© 2026 Mekong Coffee

560px wide

HTML មានចំណងជើងប្រាំមួយកម្រិត ពី `<h1>` ដល់ `<h6>` ។ ប្រើវាដូចគ្រោងរបាយការណ៍៖ `<h1>` មួយ សម្រាប់ប្រធានបទទំព័រ `<h2>` សម្រាប់ផ្នែកធំៗ `<h3>` សម្រាប់ផ្នែករង។ កុំជ្រើសចំណងជើងដោយសារទំហំអក្សរ ព្រោះទំហំជាការងាររបស់ CSS។ អ្នកប្រើកម្មវិធីអានអេក្រង់តែងលោតពីចំណងជើងមួយទៅមួយ ដូច្នេះគ្រោងច្បាស់លាស់ក៏ជួយភាពងាយស្រួលប្រើដែរ។ អត្ថបទដាក់ក្នុង `<p>` ។ `<strong>` សម្គាល់ភាពសំខាន់ ហើយ `<em>` សម្គាល់ការសង្កត់សំឡេង។ `<br>` ចុះបន្ទាត់ ហើយ `<hr>` សម្គាល់ការបូរប្រធានបទ។ តួអក្សរពិសេសសរសេរជា entity ដូចជា `©` សម្រាប់ ©។

2.2 Links

តំណភ្ជាប់

The anchor element `<a>` turns text into a link; its `href` attribute says where the link goes. A **relative URL** such as `menu.html` points to a file next to the current page, and `../index.html` goes up one folder. An **absolute URL** such as `https://developer.mozilla.org` includes the scheme and host and works from

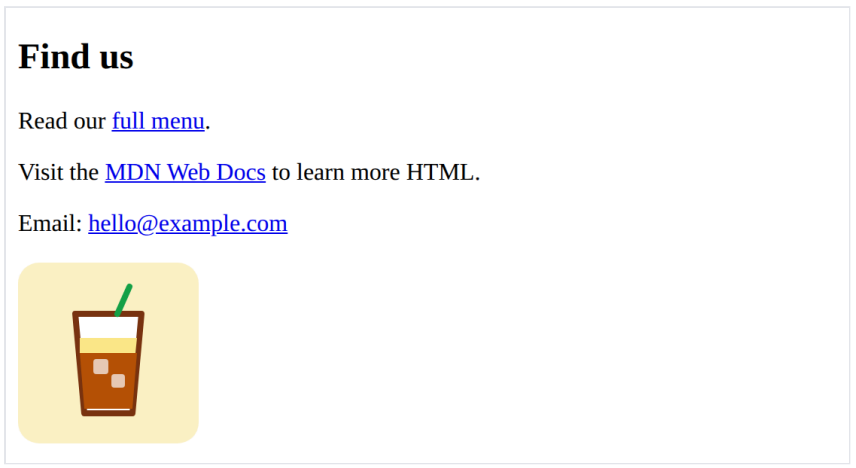
anywhere. A link to `#contact` jumps to the element with `id="contact"` on the same page, and `mailto:` opens an email program.

Link text should make sense on its own. “Read our full menu” is clear; “click here” tells a screen-reader user nothing when links are read as a list. If you open a link in a new tab with `target="_blank"`, add `rel="noopener"` so the new page cannot control the old one.

HTML INDEX.HTML LISTING 2.2 · EX/S2_LINKS

```
<h2>Find us</h2>
<p>Read our <a href="menu.html">full menu</a>.</p>
<p>Visit the
  <a href="https://developer.mozilla.org" target="_blank" rel="noopener">MDN Web
Docs</a>
  to learn more HTML.</p>
<p>Email: <a href="mailto:hello@example.com">hello@example.com</a></p>
<img src="cup.svg" alt="A cup of iced coffee with milk" width="120" height="120">
```

RESULT IN THE BROWSER · លទ្ធផល CHROMIUM



560px wide

ធាតុ `<a>` បង្កើតតំណ ហើយ `href` ប្រាប់ទិសដៅ។ **URL ទាក់ទង (relative)** ដូចជា `menu.html` ចង្អុលទៅឯកសារក្បែរទំព័របច្ចុប្បន្ន ហើយ `../index.html` ឡើងមួយថត។ **URL ពេញ (absolute)** មាន scheme និង host ហើយដំណើរការពីគ្រប់ទីកន្លែង។ តំណទៅ `#contact` លោតទៅធាតុដែលមាន `id="contact"` ។ អត្ថបទតំណគួរតែមានន័យដោយខ្លួនឯង៖ «អានម៉ឺនុយពេញ» ច្បាស់ជាង «ចុចទីនេះ»។ ពេលប្រើ `target="_blank"` សូមបន្ថែម `rel="noopener"` ។

2.3 Images

រូបភាព

The `<img>` element has no end tag. Its `src` attribute gives the file and its `alt` attribute gives **alternative text**: what the image communicates, written for someone who cannot

see it. Screen readers read it aloud, search engines index it, and browsers show it if the image fails to load. Write what matters in context (“A cup of iced coffee with milk”), not “image” or the file name. If an image is pure decoration, use an empty `alt=""` so screen readers skip it [6].

Always give `width` and `height`. The browser can then reserve space before the image arrives, so the text does not jump while the page loads. Choose the right format: **JPEG** or **WebP** for photos, **PNG** for screenshots with sharp edges, and **SVG** for logos and icons, which stay sharp at any size.

> ធាតុ `<img>` គ្មានស្លាកបិទទេ។ `src` ប្រាប់ឯកសារ ហើយ `alt` ផ្តល់ **អត្ថបទជំនួស**៖ អ្វីដែលរូបភាពចង់ប្រាប់ សរសេរសម្រាប់អ្នកដែលមើលមិនឃើញ។ កម្មវិធីអានអេក្រង់អានវាឮៗ ម៉ាស៊ីនស្វែងរកប្រើវា ហើយ browser បង្ហាញវាបើរូបភាពផ្ទុកមិនបាន។ បើរូបភាពគ្រាន់តែជាការតុបតែង សូមប្រើ `alt=""` ទទេ។ ត្រូវដាក់ `width` និង `height` ជានិច្ច ដើម្បីកុំឱ្យអត្ថបទលោតពេលទំព័រកំពុងផ្ទុក។ ប្រើ JPEG ឬ WebP សម្រាប់រូបថត PNG សម្រាប់រូបថតអេក្រង់ និង SVG សម្រាប់ឡូហ្គោ និងរូបតំណាង។

2.4 Lists

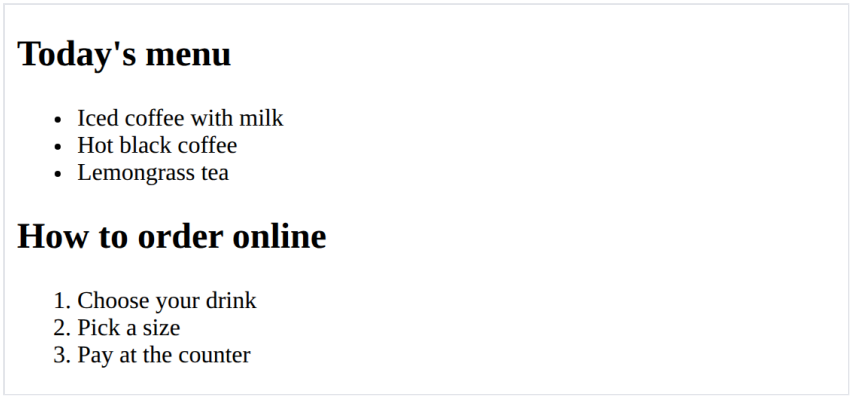
បញ្ជី

Use `<ul>` (unordered list) when order does not matter, such as a menu, and `<ol>` (ordered list) when it does, such as steps. Each item is an `<li>`. Lists can be nested: put a complete `<ul>` inside an `<li>`. Navigation menus are usually lists of links, because a menu really is a list.

HTML INDEX.HTML LISTING 2.3 · EX/S2_LISTS

```html
<h2>Today's menu</h2>
<ul>
  <li>Iced coffee with milk</li>
  <li>Hot black coffee</li>
  <li>Lemongrass tea</li>
</ul>
<h2>How to order online</h2>
<ol>
  <li>Choose your drink</li>
  <li>Pick a size</li>
  <li>Pay at the counter</li>
</ol>
```

RESULT IN THE BROWSER · លទ្ធផល CHROMIUM



560px wide

ប្រើ `<ul>` ពេលលំដាប់មិនសំខាន់ ដូចជាម៉ឺនុយភេសជ្ជៈ និង `<ol>` ពេលលំដាប់សំខាន់ ដូចជាជំហាន។ ធាតុនីមួយៗជា `<li>` ។ បញ្ជីអាចនៅក្នុងបញ្ជីបាន ដោយដាក់ `<ul>` ពេញលេញក្នុង `<li>` ។ ម៉ឺនុយនារីហ្គេសិនជាធម្មតាជាបញ្ជីតំណ។

2.5 Valid, clean HTML

HTML ត្រឹមត្រូវ និងស្អាត

Browsers are forgiving: they try to show even broken HTML. That is why errors can hide for months and then break a page in another browser. Check your pages with the free W3C Markup Validation Service (validator.w3.org), which lists missing end tags, duplicate `id` values and invalid attributes. For a gentle, illustrated introduction to HTML and CSS, see Duckett [7]. Indent nested elements by two spaces, use lowercase tag names, put attribute values in double quotes, and add short comments, `<!-- like this -->`, where the structure is not obvious.

Browser អត់ឱនឱ្យកំហុស៖ វាព្យាយាមបង្ហាញសូម្បីតែ HTML ខូច។ ហេតុនេះកំហុសអាចលាក់ខ្លួនរាប់ខែ ហើយបំបែកទំព័រនៅក្នុង browser ផ្សេង។ ពិនិត្យទំព័រជាមួយ W3C Markup Validation Service (validator.w3.org)។ ចូលបន្ទាត់ធាតុដែលនៅខាងក្នុងពីរដកឃ្លា ប្រើអក្សរតូចសម្រាប់ឈ្មោះស្លាក ដាក់តម្លៃ attribute ក្នុងសញ្ញាសម្រង់ទ្វេ ហើយសរសេរ comment ខ្លីៗ។

IN THE AI ERA ក្នុងយុគសម័យ AI

AI tools are good at producing HTML quickly, and also at producing **"div soup"**: pages built only from `<div>` and `<span>`, with images that lack `alt` text. After any AI draft, check three things: Is there one clear `<h1>` and a logical heading order? Does every meaningful image have a real `alt`? Does the validator report zero errors? A useful prompt: "Review my HTML for semantics and accessibility. List the problems; do not rewrite the file."

ឧបករណ៍ AI ពូកែបង្កើត HTML លឿន ប៉ុន្តែក៏តែងបង្កើត «div soup» ដែលប្រើតែ `<div>` ហើយរូបភាពគ្មាន alt។ ក្រោយពេលទទួលសេចក្តីព្រាងពី AI សូមពិនិត្យបីចំណុច៖ មាន h1 តែមួយ និងលំដាប់ចំណងជើងសមហេតុផល? រូបភាពមានអត្ថន័យមាន alt ពិត? validator រាយការណ៍សូន្យកំហុស?

LAB 2 · YOUR "ABOUT ME" PAGE Lab ២ · ទំព័រ «អំពីខ្ញុំ»

1. In `portfolio/`, create `about.html` with a proper head, one `<h1>` and three `<h2>` sections: Education, Skills, Hobbies.
   បង្កើត about.html ដែលមាន h1 មួយ និង h2 បី។
2. Add a photo or an illustration you have the right to use, with meaningful `alt` text and width and height.
   បន្ថែមរូបភាពដែលអ្នកមានសិទ្ធិប្រើ ជាមួយ alt ដែលមានន័យ។
3. Add an unordered list of skills and an ordered list of your top three goals for this course.
   បន្ថែមបញ្ជីជំនាញ និងបញ្ជីគោលដៅបីដែលមានលេខ។
4. Link `index.html` and `about.html` to each other, and add one external link with `rel="noopener"`.
   ភ្ជាប់ទំព័រទាំងពីរទៅគ្នា និងបន្ថែមតំណខាងក្រៅមួយ។
5. Run both pages through the W3C validator until there are no errors.
   ពិនិត្យទំព័រទាំងពីរជាមួយ validator រហូតគ្មានកំហុស។

Exercises

លំហាត់

1. Why is it wrong to use `<h4>` just because you want smaller text?
   ហេតុអ្វីខុស បើប្រើ h4 គ្រាន់តែចង់បានអក្សរតូច?
2. Your page is `/blog/post.html`. Write a relative link from it to `/index.html`.
   សរសេរតំណទាក់ទងពី /blog/post.html ទៅ /index.html។
3. Write good `alt` text for: (a) a logo that links to the home page, (b) a decorative wave line, (c) a chart showing sales rising from January to June.
   សរសេរ alt សម្រាប់រូបភាពទាំងបី។
4. Mark up a recipe with an ingredient list and numbered steps.
   សរសេររូបមន្តធ្វើមួបជាមួយបញ្ជីគ្រឿងផ្សំ និងជំហានមានលេខ។
5. Find two errors: `<p>Open <strong>daily</p></strong>` and `<img src="a.jpg">`.
   រកកំហុសពីរក្នុងកូដនេះ។

SESSION 3

SEMANTIC HTML, TABLES AND FORMS

HTML មានអត្ថន័យ តារាង និងទម្រង់

IN THIS SESSION ក្នុងវគ្គនេះ

- Structure a page with landmark elements instead of plain boxes
- Build data tables that screen readers can understand
- Create forms with labels, the right input types and validation
- Explain why browser validation never replaces server checks
- រៀបចំទំព័រដោយធាតុសម្គាល់តំបន់ (landmark) ជំនួសប្រអប់ធម្មតា
- បង្កើតតារាងទិន្នន័យដែលកម្មវិធីអានអេក្រង់យល់បាន
- បង្កើតទម្រង់ (form) ដែលមាន label ប្រភេទ input ត្រឹមត្រូវ និងការផ្ទៀងផ្ទាត់
- ពន្យល់ថាហេតុអ្វីការផ្ទៀងផ្ទាត់ក្នុង browser មិនអាចជំនួសការពិនិត្យលើ server

3.1 Semantic elements

ធាតុមានអត្ថន័យ

Semantic means “with meaning”. A `<div>` is a generic box that says nothing about its content. Semantic elements describe their role, so browsers, search engines and assistive technology can understand the page. The most important ones mark the main regions, or **landmarks**, of a page [3]:

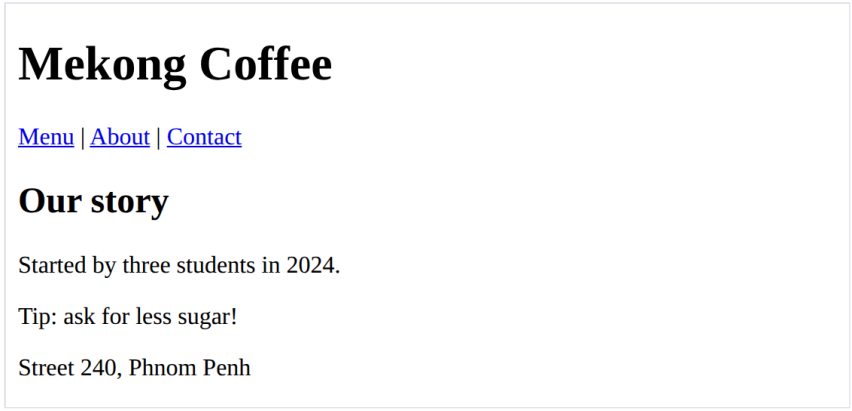
Element	Use it for	ប្រើសម្រាប់
`<header>`	Introductory content: logo, site title, often navigation	ផ្នែកក្បាល៖ ឡូហ្គោ ចំណងជើង
`<nav>`	Main groups of navigation links	ក្រុមតំណនាវីហ្គេសិនសំខាន់ៗ
`<main>`	The unique main content; only one per page	មាតិកាចម្បង (មួយក្នុងមួយទំព័រ)
`<article>`	Self-contained content: a post, a product, a news story	មាតិកាឯករាជ្យ៖ អត្ថបទ ផលិតផល
`<section>`	A themed group of content, normally with a heading	ក្រុមមាតិកាតាមប្រធានបទ មានចំណងជើង

`<aside>`	Related but secondary content: tips, sidebars	មាតិកាបន្ទាប់បន្សំ៖ គន្លឹះ របារចំហៀង
`<footer>`	Closing information: contact, copyright	ព័ត៌មានបញ្ចប់៖ ទំនាក់ទំនង រក្សាសិទ្ធិ

HTML INDEX.HTML LISTING 3.1 · EX/S3_SEMANTIC

```
<header>
  <h1>Mekong Coffee</h1>
  <nav>
    <a href="#menu">Menu</a> |
    <a href="#about">About</a> |
    <a href="#contact">Contact</a>
  </nav>
</header>
<main>
  <article id="about">
    <h2>Our story</h2>
    <p>Started by three students in 2024.</p>
  </article>
  <aside>
    <p>Tip: ask for less sugar!</p>
  </aside>
</main>
<footer>
  <p>Street 240, Phnom Penh</p>
</footer>
```

RESULT IN THE BROWSER · លទ្ធផល CHROMIUM



560px wide

The page looks almost unstyled, and that is the point: semantic HTML is about meaning, not looks. A screen-reader user can now jump straight to the navigation or the main content. Use `<div>` only when no semantic element fits, usually as a hook for CSS layout.

Semantic មានន័យថា «មានអត្ថន័យ»។ `<div>` ជាប្រអប់ទូទៅដែលមិនប្រាប់អ្វីពីមាតិកា។ ធាតុមានអត្ថន័យពិពណ៌នាតួនាទីរបស់វា ដូច្នេះ browser ម៉ាស៊ីនស្វែងរក និងបច្ចេកវិទ្យាជំនួយយល់ពីទំព័រ។ ធាតុសំខាន់បំផុតសម្គាល់តំបន់ចម្បង (landmark)៖ header, nav, main, article, section, aside និង footer។ ទំព័រមើលទៅស្ទើរតែគ្មានរចនាប័ទ្ម ព្រោះ HTML មានអត្ថន័យនិយាយពីអត្ថន័យ មិនមែនរូបរាង។ អ្នកប្រើកម្មវិធីអានអេក្រង់អាចលោតទៅកាន់ nav ឬ main ភ្លាមៗ។ ប្រើ `<div>` តែពេលគ្មានធាតុមានអត្ថន័យណាសម។

3.2 Tables for data

តារាងសម្រាប់ទិន្នន័យ

Use `<table>` for **tabular data**, information with rows and columns such as prices or timetables, never for page layout. A table has rows `<tr>`, header cells `<th>` and data cells `<td>`. Group the header row in `<thead>` and the data in `<tbody>`, give the table a `<caption>`, and use `scope="col"` or `scope="row"` so a screen reader can announce “Large, Iced coffee, $2.00” instead of just “$2.00”.

HTML INDEX.HTML LISTING 3.2 · EX/S3_TABLE

```
<style>
  table{border-collapse:collapse}
  th,td{border:1px solid #999;padding:6px 10px}
  caption{font-weight:bold;margin-bottom:6px}
</style>

<table>
  <caption>Drink prices</caption>
  <thead>
    <tr><th scope="col">Drink</th><th scope="col">Small</th><th
scope="col">Large</th></tr>
  </thead>
  <tbody>
    <tr><th scope="row">Iced coffee</th><td>$1.50</td><td>$2.00</td></tr>
    <tr><th scope="row">Green tea</th><td>$1.25</td><td>$1.75</td></tr>
  </tbody>
</table>
```

RESULT IN THE BROWSER · លទ្ធផល CHROMIUM

Drink prices

Drink	Small	Large
Iced coffee	$1.50	$2.00
Green tea	$1.25	$1.75

560px wide

> ប្រើ `<table>` សម្រាប់ **ទិន្នន័យជាតារាង** ដូចជាតម្លៃ ឬកាលវិភាគ មិនមែនសម្រាប់ប្លង់ទំព័រទេ។ តារាងមានជួរដេក `<tr>` ក្រឡាចំណងជើង `<th>` និងក្រឡាទិន្នន័យ `<td>` ។ ដាក់ជួរចំណងជើងក្នុង `<thead>` ទិន្នន័យក្នុង `<tbody>` ផ្តល់ `<caption>` ហើយប្រើ `scope` ដើម្បីឱ្យកម្មវិធីអានអេក្រង់អាន «Large, Iced coffee, $2.00» ជំនួសឱ្យ «$2.00» តែម្យ៉ាង។

3.3 Forms

ទម្រង់បញ្ចូលទិន្នន័យ

Forms let users send information: a search, a login, an order. The `<form>` element has an `action` (where to send the data) and a `method`: `get` puts the data in the URL and suits searches, while `post` sends it in the request body and suits anything that changes data or is private. Every input needs a `name`, which becomes the key in the submitted data.

Every input also needs a visible `<label>` whose `for` attribute matches the input's `id`. Clicking the label then focuses the input, which helps everyone on a small touch screen, and a screen reader reads the label when the input gets focus. A placeholder is not a label: it disappears as soon as the user types.

HTML INDEX.HTML LISTING 3.3 · EX/S3_FORM

```html
<style>
  label{display:block;margin-top:10px}
  input,select,textarea{font:inherit}
</style>

<form action="/order" method="post">
  <label for="name">Your name</label>
  <input id="name" name="name" type="text" required>

  <label for="email">Email</label>
  <input id="email" name="email" type="email" required>

  <label for="size">Size</label>
  <select id="size" name="size">
    <option>Small</option>
    <option>Large</option>
  </select>

  <label for="note">Note</label>
  <textarea id="note" name="note" rows="2"></textarea>

  <p><button type="submit">Place order</button></p>
</form>
```

RESULT IN THE BROWSER · លទ្ធផល CHROMIUM

Your name

Email

Size
Small

Note

Place order

560px wide

ទម្រង់ (form) អនុញ្ញាតឱ្យអ្នកប្រើផ្ញើព័ត៌មាន៖ ការស្វែងរក ការចូលគណនី ការបញ្ជាទិញ។ `<form>` មាន `action` (ផ្ញើទៅណា) និង `method` ៖ `get` ដាក់ទិន្នន័យក្នុង URL សមសម្រាប់ការស្វែងរក ចំណែក `post` ផ្ញើក្នុងតួសំណើ សមសម្រាប់ទិន្នន័យឯកជន ឬការផ្លាស់ប្តូរទិន្នន័យ។ input នីមួយៗត្រូវការ `name` និង `<label>` ដែល `for` ត្រូវនឹង `id` របស់ input។ ចុចលើ label នឹងផ្តោតលើ input ហើយកម្មវិធីអានអេក្រង់អាន label។ Placeholder មិនមែនជា label ទេ ព្រោះវាបាត់ពេលអ្នកប្រើចាប់ផ្តើមវាយ។

Input types and built-in validation · ប្រភេទ input និងការផ្ទៀងផ្ទាត់

Choosing the right `type` gives users the right keyboard on a phone and free checks: `email`, `tel`, `number`, `date`, `url`, `password`, `checkbox` and `radio`. Attributes add rules: `required`, `min` and `max`, `minlength` and `maxlength`, and `pattern` for a regular expression. If a rule fails, the browser stops the submission and shows a message. Group related radio buttons or checkboxes in a `<fieldset>` with a `<legend>`.

KEY IDEA គំនិតសំខាន់

Browser validation is for **convenience**, not **security**. Anyone can switch it off in DevTools or send data without a browser. The server must always check the data again.

ការផ្ទៀងផ្ទាត់ក្នុង browser គឺសម្រាប់ **ភាពងាយស្រួល** មិនមែន **សុវត្ថិភាព** ទេ។ នរណាក៏អាចបិទវាក្នុង DevTools ឬផ្ញើទិន្នន័យដោយមិនប្រើ browser បាន។ Server ត្រូវតែពិនិត្យទិន្នន័យម្តងទៀតជានិច្ច។

ការជ្រើស `type` ត្រឹមត្រូវផ្តល់ការចុចសមស្របលើទូរស័ព្ទ និងការពិនិត្យដោយស្វ័យប្រវត្តិ៖ email, tel, number, date, url, password, checkbox និង radio។ គុណលក្ខណៈបន្ថែមច្បាប់៖ `required`, `min` / `max`, `minlength` / `maxlength` និង `pattern`។ ដាក់ radio ឬ checkbox ដែលទាក់ទងគ្នាក្នុង `<fieldset>` ជាមួយ `<legend>`។

IN THE AI ERA ក្នុងយុគសម័យ AI

Ask an assistant to turn a paper registration form into HTML, then audit the result: every input has a matching label, the right `type`, a `name`, and sensible required fields. Ask “Which fields would a server still need to validate, and why?” This turns the AI into a reviewer and keeps the security thinking in your head.

សុំ AI បំប្លែងទម្រង់ក្រដាសទៅជា HTML រួចពិនិត្យលទ្ធផល៖ input នីមួយៗមាន label ត្រូវគ្នា type ត្រឹមត្រូវ name និងវាលចាំបាច់សមហេតុផល។ សួរថា «តើវាលណាខ្លះ server នៅតែត្រូវផ្ទៀងផ្ទាត់ ហើយហេតុអ្វី?»។

LAB 3 · STRUCTURE AND CONTACT FORM Lab ៣ · រចនាសម្ព័ន្ធ និងទម្រង់ទំនាក់ទំនង

1. Rewrite `index.html` and `about.html` with `<header>`, `<nav>`, `<main>` and `<footer>`. Put the same navigation on both pages.
   សរសេរទំព័រឡើងវិញជាមួយ header, nav, main, footer។
2. Add a `projects.html` page with a table of three course projects: name, date, technology.

បង្កើត projects.html ដែលមានតារាងគម្រោងបី។

3. Create `contact.html` with name, email, a topic chosen by radio buttons in a fieldset, and a message. Make sensible fields required.
   បង្កើត contact.html ដែលមានឈ្មោះ អ៊ីមែល ប្រធានបទ (radio) និងសារ។
4. Test the form: submit it empty, then with a wrong email. Then open DevTools, remove `required` from one input and submit again. What does this prove?
   សាកល្បងទម្រង់ ហើយលុប required ក្នុង DevTools។ តើវាបង្ហាញអ្វី?
5. Navigate the whole site using only the Tab and Enter keys. Note anything you cannot reach.
   ប្រើតែគ្រាប់ចុច Tab និង Enter ដើម្បីរុករកគេហទំព័រទាំងមូល។

Exercises

លំហាត់

1. Choose the best element: (a) a blog post, (b) the site menu, (c) a “related products” box, (d) the copyright line.
   ជ្រើសធាតុល្អបំផុតសម្រាប់ករណីទាំងបួន។
2. Why should tables not be used for page layout?
   ហេតុអ្វីមិនគួរប្រើតារាងសម្រាប់ប្លង់ទំព័រ?
3. Write a form for a student ID (exactly 8 digits) using `pattern`.
   សរសេរ input សម្រាប់អត្តលេខនិស្សិត ៨ ខ្ទង់ ដោយប្រើ pattern។
4. When should a form use `get` and when `post`? Give one example of each.
   ពេលណាប្រើ get និងពេលណាប្រើ post?
5. A classmate uses placeholders instead of labels. Explain two problems this causes.
   ពន្យល់បញ្ហាពីរនៃការប្រើ placeholder ជំនួស label។

SESSION 4

DESIGN PRINCIPLES, UX AND ACCESSIBILITY

គោលការណ៍រចនា UX និងភាពងាយស្រួលប្រើសម្រាប់មនុស្សគ្រប់គ្នា

IN THIS SESSION ក្នុងវគ្គនេះ

- Follow a simple design process from research to testing
- Apply hierarchy, contrast, alignment, proximity and repetition
- Use usability heuristics to review an interface
- Check colour contrast and other key accessibility rules from WCAG 2.2

- អនុវត្តដំណើរការរចនាសាមញ្ញ ពីការស្រាវជ្រាវដល់ការសាកល្បង
- អនុវត្តហានានុក្រម កម្រិតពណ៌ផ្ទុយ ការតម្រឹម ភាពជិតគ្នា និងការធ្វើម្តងទៀត
- ប្រើគោលការណ៍ងាយស្រួលប្រើ (heuristics) ដើម្បីពិនិត្យផ្ទៃប្រើប្រាស់
- ពិនិត្យកម្រិតពណ៌ផ្ទុយ និងច្បាប់សំខាន់ៗរបស់ WCAG 2.2

4.1 A simple design process

ដំណើរការរចនាសាមញ្ញ

Good design starts with people, not with colours. Before you open your editor, answer three questions: Who will use this site? What do they want to do? What does the owner want them to do? Then move through four short steps and repeat them as you learn more.

DISCOVER
Talk to users, list their goals and the content they need.
ស្វែងយល់អ្នកប្រើ និងគោលដៅរបស់ពួកគេ

WIREFRAME
Sketch the layout in grey boxes, on paper first.
គូរប្លង់ជាប្រអប់ពណ៌ប្រផេះ

PROTOTYPE
Build a clickable version in HTML or a design tool.
បង្កើតគំរូដែលអាចចុចបាន

TEST
Watch real people try it. Fix, then test again.
មើលមនុស្សពិតសាកល្បង ហើយកែលម្អ

> ការរចនាល្អចាប់ផ្តើមពីមនុស្ស មិនមែនពីពណ៌ទេ។ មុនពេលបើកកម្មវិធីសរសេរកូដ សូមឆ្លើយសំណួរបី៖ តើនរណាប្រើគេហទំព័រនេះ? ពួកគេចង់ធ្វើអ្វី? ម្ចាស់គេហទំព័រចង់ឱ្យពួកគេធ្វើអ្វី? បន្ទាប់មកធ្វើតាមជំហានខ្លីៗបួន៖ ស្វែងយល់

> គូរប្លង់ព្រាង (wireframe) បង្កើតគំរូ (prototype) និងសាកល្បង ហើយធ្វើវាម្តងទៀតពេលអ្នករៀនបានច្រើនជាងមុន។

4.2 Five visual principles

គោលការណ៍មើលឃើញច្រាំ

Hierarchy shows what matters most through size, weight, colour and position, so the eye knows where to start. **Contrast** makes different things look clearly different. **Alignment** puts elements on shared invisible lines, which makes a page feel calm and organised. **Proximity** groups related items close together and separates unrelated ones with space. **Repetition** reuses the same styles for the same kinds of things, so users learn your interface once. White space is not wasted space: it is what makes grouping and hierarchy visible [8].

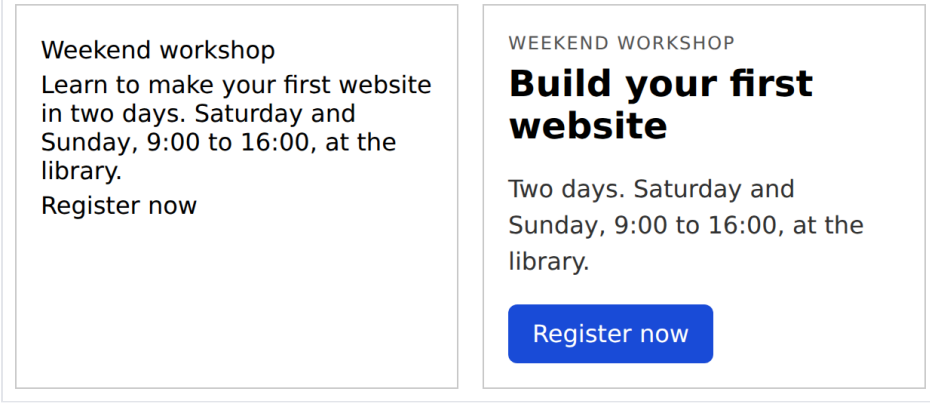
Listing 4.1 shows the same workshop announcement twice. The content is identical; only hierarchy, spacing and one strong call-to-action button differ.

CSS STYLE.CSS LISTING 4.1 · EX/S4_HIERARCHY

```css
body { font-family: system-ui, sans-serif; display: flex; gap: 16px; }
.card { width: 260px; padding: 16px; border: 1px solid #ccc; }
.plain p { margin: 4px 0; }              /* everything looks the same */

.designed .label { font-size: 12px; text-transform: uppercase;
                   letter-spacing: .08em; color: #555; margin: 0; }
.designed h2 { font-size: 24px; margin: 4px 0 8px; }
.designed p  { color: #333; line-height: 1.5; }
.btn { display: inline-block; background: #1d4ed8; color: #fff;
       padding: 10px 16px; border-radius: 6px; text-decoration: none; }
```

RESULT IN THE BROWSER · លទ្ធផល CHROMIUM



620px wide

ហានានុក្រម បង្ហាញអ្វីដែលសំខាន់បំផុតតាមទំហំ កម្រាស់ ពណ៌ និងទីតាំង។ **ភាពផ្ទុយ** ធ្វើឱ្យអ្វីដែលខុសគ្នាមើលទៅខុសគ្នាច្បាស់។ **ការតម្រឹម** ដាក់ធាតុលើបន្ទាត់មើលមិនឃើញរួមគ្នា ធ្វើឱ្យទំព័រមើលទៅមានរបៀប។ **ភាពជិតគ្នា** ដាក់ធាតុដែលទាក់ទងគ្នាជិតគ្នា។ **ការធ្វើឡើងវិញ** ប្រើរចនាប័ទ្មដដែលសម្រាប់របស់ប្រភេទដូចគ្នា។ ចន្លោះទទេ (white space) មិនមែនខ្ជះខ្ជាយទេ វាធ្វើឱ្យការដាក់ជាក្រុម និងហានានុក្រមមើលឃើញ។ Listing 4.1 បង្ហាញការប្រកាសដដែលពីរដង៖ មាតិកាដូចគ្នា ខុសគ្នាតែហានានុក្រម គម្លាត និងប៊ូតុងសកម្មភាពមួយ។

Typography and colour · ពុម្ពអក្សរ និងពណ៌

Use at most two font families, for example one for headings and one for body text. Body text on screens is usually 16 px or larger with a line height around 1.5, and lines of about 45 to 75 characters are comfortable to read [9]. For Khmer, choose a font designed for the script, such as Noto Sans Khmer or Kantumruy Pro, and give it a slightly larger line height, because Khmer has stacked subscript consonants and vowels above and below the line.

For colour, start small: one neutral for text, one light background, one brand colour for links and buttons, and one colour for errors. A common rule of thumb is about 60% neutral background, 30% secondary colour and 10% accent. Never use colour as the only signal: an error should also have a text message or an icon.

ប្រើពុម្ពអក្សរយ៉ាងច្រើនពីរ ឧទាហរណ៍មួយសម្រាប់ចំណងជើង និងមួយសម្រាប់អត្ថបទ។ អត្ថបទលើអេក្រង់ជាធម្មតាមានទំហំ 16 px ឬធំជាងនេះ កម្ពស់បន្ទាត់ប្រហែល 1.5 ហើយបន្ទាត់ប្រហែល 45 ទៅ 75 តួអក្សរងាយស្រួលអាន។ សម្រាប់អក្សរខ្មែរ សូមជ្រើសពុម្ពអក្សរដែលរចនាសម្រាប់អក្សរខ្មែរ ដូចជា Noto Sans Khmer ឬ Kantumruy Pro ហើយផ្តល់កម្ពស់បន្ទាត់ធំជាងបន្តិច ព្រោះអក្សរខ្មែរមានជើង និងស្រៈនៅខាងលើ និងខាងក្រោម។ សម្រាប់ពណ៌ ចាប់ផ្តើមតិចៗ៖ ពណ៌អព្យាក្រឹតសម្រាប់អត្ថបទ ផ្ទៃខាងក្រោយភ្លឺ ពណ៌ម៉ាកមួយសម្រាប់តំណ និងប៊ូតុង និងពណ៌មួយសម្រាប់កំហុស។ កុំប្រើពណ៌ជាសញ្ញាតែមួយគត់។

4.3 Usability heuristics

គោលការណ៍ងាយស្រួលប្រើ

Jakob Nielsen's ten usability heuristics are a quick checklist for reviewing any interface [10]. Five that matter most for a first website are:

- **Visibility of system status**: show what is happening, such as "Sending…" after a click.
- **Match the real world**: use the users' words, not internal jargon.
- **Consistency and standards**: links look like links; the logo goes home.
- **Error prevention and recovery**: prevent mistakes, and explain errors in plain language with a way to fix them.
- **Recognition rather than recall**: keep options visible instead of making users remember them.

គោលការណ៍ងាយស្រួលប្រើទាំងដប់របស់ Jakob Nielsen ជាបញ្ជីត្រួតពិនិត្យរហ័សសម្រាប់ពិនិត្យផ្ទៃប្រើប្រាស់។ ប្រាំដែលសំខាន់បំផុតសម្រាប់គេហទំព័រដំបូង៖ បង្ហាញស្ថានភាពប្រព័ន្ធ (ដូចជា «កំពុងផ្ទើ...») ប្រើពាក្យរបស់អ្នកប្រើ ភាពស៊ីសង្វាក់ និងស្តង់ដារ ការការពារ និងការជួសជុលកំហុសដោយពន្យល់ជាភាសាសាមញ្ញ និងការមើលឃើញជំនួសការចងចាំ។

4.4 Accessibility and WCAG 2.2

ភាពងាយស្រួលប្រើ និង WCAG 2.2

Accessibility means people with disabilities can perceive, understand, navigate and use a site. It also helps everyone: people on a phone in bright sunlight, older users and people with a broken arm. The international standard is the Web Content Accessibility Guidelines, **WCAG 2.2**, a W3C Recommendation published on 5 October 2023 [6]. It is built on four principles: content must be **Perceivable**, **Operable**, **Understandable** and **Robust** (POUR). Most organisations aim for level AA.

KEY IDEA · WCAG 2.2 LEVEL AA EXAMPLES ឧទាហរណ៍កម្រិត AA

Text contrast of at least **4.5:1**, or **3:1** for large text (about 24 px, or 18.5 px bold) [11]. **Keyboard**: everything works without a mouse, and focus is visible. **Target size**: click and tap targets of at least **24 by 24 CSS pixels**, with some exceptions [12]. **Text alternatives** for meaningful images, and **labels** for form inputs.

កម្រិតពណ៌ផ្ទុយយ៉ាងតិច 4.5:1 (ឬ 3:1 សម្រាប់អក្សរធំ)។ អ្វីៗដំណើរការដោយក្តារចុច ហើយការផ្តោត (focus) មើលឃើញ។ គោលដៅចុចយ៉ាងតិច 24 × 24 CSS pixel។ អត្ថបទជំនួសសម្រាប់រូបភាព និង label សម្រាប់ input។

The **contrast ratio** compares the relative brightness of text and background, from 1:1 (no contrast) to 21:1 (black on white). Free checkers do the maths for you. The ratios below were calculated with the WCAG formula.

Text colour	On	Ratio	Normal text (4.5:1)
`#1F2937` dark grey	white	14.68:1	Pass
`#6B7280` mid grey	white	4.83:1	Pass, just
`#9CA3AF` light grey	white	2.54:1	Fail
white	`#1D4ED8` blue	6.70:1	Pass
`#FBBF24` yellow	white	1.67:1	Fail

ភាពងាយស្រួលប្រើ (accessibility) មានន័យថាជនពិការអាចយល់ រុករក និងប្រើគេហទំព័របាន។ វាក៏ជួយមនុស្សគ្រប់គ្នាដែរ ដូចជាអ្នកប្រើទូរស័ព្ទក្រោមពន្លឺថ្ងៃ ឬមនុស្សចាស់។ ស្តង់ដារអន្តរជាតិគឺ **WCAG 2.2** ដែល W3C បានចេញផ្សាយថ្ងៃទី ៥ ខែតុលា ឆ្នាំ ២០២៣។ វាផ្អែកលើគោលការណ៍បួន៖ អាចយល់ឃើញ អាចប្រើបាន អាចយល់បាន និងរឹងមាំ (POUR)។ **អនុបាតពណ៌ផ្ទុយ** ប្រៀបធៀបពន្លឺនៃអក្សរ និងផ្ទៃខាងក្រោយ ពី 1:1 ដល់ 21:1។ តារាងខាងលើគណនាដោយរូបមន្ត WCAG៖ ពណ៌ប្រផេះស្រាល និងពណ៌លឿងលើផ្ទៃសមិនឆ្លងកាត់ទេ។

IN THE AI ERA ក្នុងយុគសម័យ AI

AI tools can suggest colour palettes, write UX copy and critique a screenshot of your layout. They are fast at producing ideas, but they often produce designs that look generic, and they can be confident about contrast without measuring it. Always check contrast with a real checker, test with real users, and read the licence of any AI image tool before using its images: rules on ownership and commercial use differ between tools and countries.

ឧបករណ៍ AI អាចស្នើបន្សំពណ៌ សរសេរអត្ថបទ UX និងរិះគន់រូបថតអេក្រង់ប្លង់របស់អ្នក។ ប៉ុន្តែវាតែងបង្កើតរចនាដែលមើលទៅធម្មតា។ ហើយអាចនិយាយយ៉ាងជឿជាក់ពីកម្រិតពណ៌ផ្ទុយដោយមិនបានវាស់។ សូមពិនិត្យកម្រិតពណ៌ផ្ទុយជាមួយឧបករណ៍ពិត សាកល្បងជាមួយអ្នកប្រើពិត ហើយអានអាជ្ញាប័ណ្ណរបស់ឧបករណ៍បង្កើតរូបភាព AI មុនពេលប្រើរូបភាព។

LAB 4 · WIREFRAME AND STYLE GUIDE Lab ៤ · ប្លង់ព្រាង និងមគ្គុទ្ទេសក៍រចនាប័ទ្ម

1. Write a one-paragraph brief: who will visit your portfolio (for example an employer or a scholarship committee) and what you want them to do.
   សរសេរសេចក្តីសង្ខេប៖ នរណានឹងមើលផលប័ត្ររបស់អ្នក ហើយអ្នកចង់ឱ្យពួកគេធ្វើអ្វី។
2. Sketch a wireframe of your home page for a phone and for a laptop, using grey boxes only.
   គូរប្លង់ព្រាងទំព័រដើមសម្រាប់ទូរស័ព្ទ និងកុំព្យូទ័រ។
3. Make a style guide: two fonts, four colours with hex codes, and the contrast ratio of each text and background pair. All body text must reach 4.5:1.
   បង្កើតមគ្គុទ្ទេសក៍រចនាប័ទ្ម៖ ពុម្ពអក្សរពីរ ពណ៌បួន និងអនុបាតពណ៌ផ្ទុយ។
4. Run a five-second test with two classmates: show your wireframe for five seconds and ask what the page is about.
   ធ្វើតេស្តប្រាំវិនាទីជាមួយមិត្តរួមថ្នាក់ពីរនាក់។

Exercises

លំហាត់

1. Pick a Cambodian website you use. Find one good and one bad example of hierarchy.
   ជ្រើសគេហទំព័រកម្ពុជាមួយ ហើយរកឧទាហរណ៍ល្អ និងអាក្រក់នៃឋានានុក្រម។
2. Grey text `#9CA3AF` on white fails. Suggest a fix that keeps a soft grey look.
   ពណ៌ #9CA3AF លើផ្ទៃសមិនឆ្លងកាត់។ ស្នើការកែ។
3. Which heuristic does a form break if it clears all fields after one error?
   តើទម្រង់ដែលលុបវាលទាំងអស់ក្រោយកំហុសមួយ បំពានគោលការណ៍ណា?
4. Explain POUR with one example for each letter.
   ពន្យល់ POUR ដោយឧទាហរណ៍មួយសម្រាប់អក្សរនីមួយៗ។
5. Why should an error message not rely on red colour alone?
   ហេតុអ្វីសារកំហុសមិនគួរពឹងតែលើពណ៌ក្រហម?

SESSION 5

CSS FOUNDATIONS: SELECTORS, CASCADE, TYPE AND COLOUR

មូលដ្ឋាន CSS៖ Selector, Cascade, ពុម្ពអក្សរ និងពណ៌

IN THIS SESSION ក្នុងវគ្គនេះ

- Connect an external stylesheet and write CSS rules
- Target elements with type, class, id and combined selectors
- Predict which rule wins using the cascade and specificity
- Choose units and colour formats, and style readable text
- ភ្ជាប់ឯកសាររចនាប័ទ្មខាងក្រៅ និងសរសេរច្បាប់ CSS
- ជ្រើសធាតុដោយ selector ប្រភេទ class, id និងការបន្សំ
- ទស្សន៍ទាយច្បាប់ណាឈ្នះ ដោយប្រើ cascade និង specificity
- ជ្រើសឯកតា និងទម្រង់ពណ៌ ហើយរចនាអត្ថបទឱ្យងាយអាន

5.1 Adding CSS to a page

ការបន្ថែម CSS ទៅទំព័រ

CSS can live in three places: in a `style` attribute on one element (inline), in a `<style>` element in the head (internal), or in a separate `.css` file (external). Use an external file for real sites. One stylesheet can style every page, the browser downloads it once and caches it, and your HTML stays clean. Connect it in the head:

```
<link rel="stylesheet" href="style.css">
```

A CSS **rule** has a **selector**, which chooses elements, and a **declaration block** of **property: value;** pairs. In `h1 { color: navy; font-size: 2rem; }`, `h1` is the selector, `color` and `font-size` are properties, and `navy` and `2rem` are values [13].

> CSS អាចដាក់បានបីកន្លែង៖ ក្នុង attribute `style` លើធាតុមួយ (inline) ក្នុង `<style>` ក្នុង head (internal) ឬក្នុងឯកសារ .css ដាច់ដោយឡែក (external)។ សម្រាប់គេហទំព័រពិត សូមប្រើឯកសារខាងក្រៅ៖ ឯកសារមួយអាចរចនាទំព័រទាំងអស់ browser ទាញយកវាតែម្តង ហើយ HTML នៅស្អាត។ ច្បាប់ CSS មាន **selector** ដែលជ្រើសធាតុ និងប្លុកប្រកាសដែលមានគូ **property: value;**។

5.2 Selectors

Selector

A **type selector** (`p`) matches every element of that type. A **class selector** (`.note`) matches every element with that class; classes are reusable and are the main tool of everyday CSS. An **id selector** (`#title`) matches the one element with that id. A **descendant selector** (`.menu li`) matches `li` elements anywhere inside `.menu`. You can also combine selectors (`li.new`) or list several with commas (`h1, h2`).

CSS STYLE.CSS LISTING 5.1 · EX/S5_SELECTORS

```css
/* type selector: every <p> */
p { color: #333; }

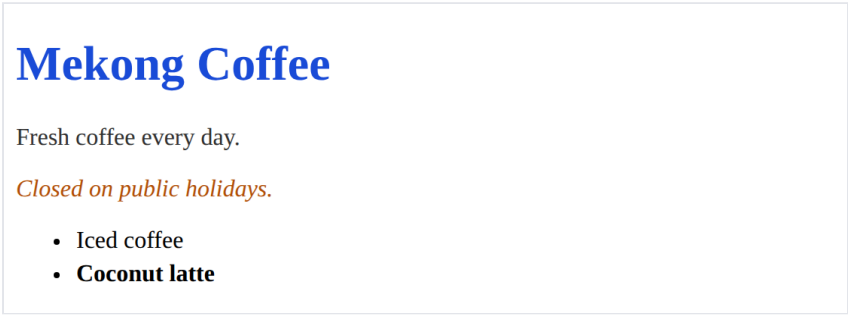
/* class selector: elements with class="note" */
.note { color: #b45309; font-style: italic; }

/* id selector: the one element with id="title" */
#title { color: #1d4ed8; }

/* descendant selector: <li> inside .menu */
.menu li { margin-bottom: 4px; }

/* two classes on one element */
li.new { font-weight: bold; }
```

RESULT IN THE BROWSER · លទ្ធផល CHROMIUM



560px wide

Pseudo-classes select elements in a certain state, such as `a:hover`, `input:focus` or `li:first-child`. You will use them for interactive states in Session 8.

Type selector (`p`) ជ្រើសធាតុទាំងអស់ប្រភេទនោះ។ **Class selector** (`.note`) ជ្រើសធាតុទាំងអស់ដែលមាន class នោះ ហើយ class អាចប្រើឡើងវិញបាន ដូច្នេះវាជាឧបករណ៍ចម្បងនៃ CSS ប្រចាំថ្ងៃ។ **Id selector** (`#title`) ជ្រើសធាតុតែមួយដែលមាន id នោះ។ **Descendant selector** (`.menu li`) ជ្រើស li ទាំងអស់នៅក្នុង .menu។ **Pseudo-class** ជ្រើសធាតុក្នុងស្ថានភាពជាក់លាក់ ដូចជា `a:hover` ឬ `input:focus`។

5.3 The cascade and specificity

Cascade និង Specificity

When several rules set the same property on the same element, the browser must choose one. This is the **cascade**, the “C” in CSS. In simple terms it decides in this order: first **importance** (avoid `!important`), then **specificity**, then **source order**, where the later rule wins.

Specificity is a score written as three numbers, id–class–type. `p` scores 0-0-1, `.intro` scores 0-1-0, `p.intro` scores 0-1-1 and `#first` scores 1-0-0. Compare from the left: one id beats any number of classes.

CSS STYLE.CSS LISTING 5.2 · EX/S5_CASCADE

```css
p        { color: green;  }   /* specificity 0-0-1 */
.intro   { color: orange; }   /* specificity 0-1-0 */
#first   { color: purple; }   /* specificity 1-0-0 */
p.intro  { font-size: 20px; }
.intro   { font-size: 14px; } /* loses: p.intro (0-1-1) is more specific */
```

RESULT IN THE BROWSER · លទ្ធផល CHROMIUM

Which colour am I?

And me?

560px wide

The first paragraph is purple because the id selector is most specific. The second paragraph is orange because `.intro` beats `p`. Both are 20 px because `p.intro` is more specific than `.intro`, even though `.intro` comes later. In DevTools, the *Styles* panel shows losing declarations crossed out, which is the fastest way to debug the cascade.

> ពេលច្បាប់ច្រើនកំណត់ property ដូចគ្នាលើធាតុតែមួយ browser ត្រូវជ្រើសមួយ។ នេះគឺ **cascade**។ វាសម្រេចតាមលំដាប់៖ ភាពសំខាន់ (ជៀសវាង !important) បន្ទាប់មក **specificity** ហើយចុងក្រោយលំដាប់ក្នុងឯកសារ (ច្បាប់ក្រោយឈ្នះ)។ Specificity ជាពិន្ទុបីខ្ទង់ id–class–type។ `p` = 0-0-1, `.intro` = 0-1-0, `p.intro` = 0-1-1, `#first` = 1-0-0។ ប្រៀបធៀបពីឆ្វេង៖ id មួយឈ្នះ class ប៉ុន្មានក៏ដោយ។ ក្នុង DevTools ផ្ទាំង *Styles* បង្ហាញការប្រកាសដែលចាញ់ដោយគូសឆូត។

KEY IDEA គំនិតសំខាន់

Keep specificity low and flat: style mostly with single classes. Avoid ids for styling and avoid `!important`; they make later changes painful. Some properties, such as `color` and `font-family`, are **inherited** by child elements; others, such as `border` and `padding`, are not.

រក្សា specificity ឱ្យទាប៖ រចនាភាគច្រើនដោយ class តែមួយ។ ជៀសវាងប្រើ id និង !important សម្រាប់រចនាប័ទ្ម។ Property ខ្លះដូចជា color និង font-family ត្រូវបាន **បន្តមរតក (inherit)** ទៅធាតុកូន ខ្លះទៀតដូចជា border និង padding មិនបន្តទេ។

5.4 Units and colours

ឯកតា និងពណ៌

Unit	Relative to	Good for
`px`	a CSS pixel	borders, small fixed details
`rem`	root font size (usually 16 px)	font sizes and spacing; respects user settings
`em`	the element's own font size	padding that grows with the text
`%`	the parent element	widths
`vw`, `vh`	1% of viewport width or height	full-screen sections
`ch`	width of the "0" character	line length, e.g. `max-width: 65ch`

Colours can be written as names (`navy`), hex codes (`#1d4ed8`), `rgb(29 78 216)` or `hsl(224 76% 48%)`. HSL (hue, saturation, lightness) is easy for humans to adjust: keep the hue and change the lightness to make a lighter or darker shade of the same colour. Add transparency with a fourth value, for example `rgb(0 0 0 / 50%)`.

ប្រើ `rem` សម្រាប់ទំហំអក្សរ និងគម្លាត ព្រោះវាគោរពការកំណត់ទំហំអក្សររបស់អ្នកប្រើ។ ប្រើ `%` សម្រាប់ទទឹង ហើយ `ch` សម្រាប់ប្រវែងបន្ទាត់។ ពណ៌អាចសរសេរជាឈ្មោះ hex, rgb() ឬ hsl()។ HSL (ពណ៌ ភាពឆ្អែត ពន្លឺ) ងាយស្រួលកែ៖ រក្សាពណ៌ ហើយប្តូរពន្លឺ ដើម្បីបានពណ៌ស្រាល ឬចាស់ជាងមុន។

5.5 Styling text

ការរចនាអត្ថបទ

Most of the web is text, so good typography is the fastest way to improve a site. Listing 5.3 sets a readable base: a font stack with fallbacks, a line height of 1.6, a maximum line length in `ch`, and sizes in `rem`. For Khmer pages, put a Khmer font first in the stack, for

example `font-family: "Noto Sans Khmer", system-ui, sans-serif;`, and load it from your own files or a font service.

CSS STYLE.CSS LISTING 5.3 · EX/S5_TYPE

```css
html { font-size: 100%; }              /* usually 16px */
body {
  font-family: Georgia, "Times New Roman", serif;
  line-height: 1.6;
  color: #222;
  max-width: 65ch;                       /* comfortable line length */
  margin: 0 auto;
  padding: 1rem;
}
h1 { font-family: system-ui, sans-serif;
     font-size: 2rem; line-height: 1.2; margin-bottom: .5rem; }
.lead { font-size: 1.25rem; color: #555; }
```

RESULT IN THE BROWSER · លទ្ធផល CHROMIUM

Learning to code in Cambodia

More students are building websites than ever before.

Good typography makes long text easy to read. Choose a comfortable size, enough line height and a line length of about 60 to 75 characters.

560px wide

ភាគច្រើននៃវេបគឺអត្ថបទ ដូច្នេះពុម្ពអក្សរល្អជាវិធីលឿនបំផុតដើម្បីកែលម្អគេហទំព័រ។ Listing 5.3 កំណត់មូលដ្ឋានងាយអាន៖ បញ្ជីពុម្ពអក្សរដែលមានជម្រើសបម្រុង កម្ពស់បន្ទាត់ 1.6 ប្រវែងបន្ទាត់អតិបរមាជា ch និងទំហំជា rem។ សម្រាប់ទំព័រខ្មែរ សូមដាក់ពុម្ពអក្សរខ្មែរមុនគេក្នុងបញ្ជី។

IN THE AI ERA ក្នុងយុគសម័យ AI

"Why is my CSS not working?" is the most common beginner question, and a good one for AI, if you give context. Paste the HTML element, the CSS rule and what you expected, then ask the assistant to explain which rule wins and why. Confirm the answer in the DevTools *Styles* and *Computed* panels. Be careful with AI suggestions that add `!important`: they fix the symptom and make the next bug harder.

«ហេតុអ្វី CSS របស់ខ្ញុំមិនដំណើរការ?» ជាសំណួរល្អសម្រាប់ AI បើអ្នកផ្តល់បរិបទ៖ បិទភ្ជាប់ធាតុ HTML ច្បាប់ CSS និងអ្វីដែលអ្នករំពឹង ហើយសុំឱ្យពន្យល់ថាច្បាប់ណាឈ្នះ និងហេតុអ្វី។ ផ្ទៀងផ្ទាត់ក្នុង DevTools។ ប្រយ័ត្នការស្នើឱ្យប្រើ !important។

LAB 5 · TYPOGRAPHY FOR YOUR PORTFOLIO Lab ៥ · ពុម្ពអក្សរសម្រាប់ផលប័ត្រ

1. Create `css/style.css` and link it from every page of your portfolio.
   បង្កើត css/style.css ហើយភ្ជាប់ពីទំព័រទាំងអស់។
2. Apply the fonts and colours from your Session 4 style guide. Use `rem` for all font sizes.
   អនុវត្តពុម្ពអក្សរ និងពណ៌ពីមគ្គុទ្ទេសក៍រចនាប័ទ្មរបស់អ្នក។
3. Style links, including `:hover` and `:focus` states.
   រចនាតំណ រួមទាំងស្ថានភាព :hover និង :focus។
4. Create three utility classes, for example `.lead`, `.muted` and `.highlight`, and use them on at least two pages.
   បង្កើត class ប្រើប្រាស់បី ហើយប្រើវាលើទំព័រយ៉ាងតិចពីរ។
5. Change your browser's default font size to "Large". Does your site scale? If not, find the `px` values to replace.
   ប្តូរទំហំអក្សរលំនាំដើមរបស់ browser។ តើគេហទំព័ររបស់អ្នកពង្រីកតាមទេ?

Exercises

លំហាត់

1. Calculate the specificity of `nav ul li a`, `.nav a` and `#main .card h2`. Which wins?
   គណនា specificity នៃ selector ទាំងបី។ តើមួយណាឈ្នះ?
2. Write one rule that gives all `h1`, `h2` and `h3` the same font.
   សរសេរច្បាប់មួយសម្រាប់ h1, h2 និង h3។
3. Why is `rem` usually better than `px` for font size?
   ហេតុអ្វី rem ល្អជាង px សម្រាប់ទំហំអក្សរ?

---

4. Write `#1d4ed8` with 50% transparency using `rgb()`. (Hint: 1d = 29, 4e = 78, d8 = 216.)
   សរសេរពណ៌នេះជាមួយភាពថ្លា 50%។
5. A paragraph is red but you wanted blue. List three things to check in DevTools.
   រាយបីចំណុចដែលត្រូវពិនិត្យក្នុង DevTools។

SESSION 6

THE BOX MODEL AND FLEXBOX

ម៉ូដែលប្រអប់ និង Flexbox

IN THIS SESSION ក្នុងវគ្គនេះ

- Explain content, padding, border and margin
- Use `box-sizing: border-box` and know why
- Tell block, inline and inline-block elements apart
- Build a header, a navigation bar and a row of cards with Flexbox
- ពន្យល់ content, padding, border និង margin
- ប្រើ box-sizing: border-box ហើយដឹងពីមូលហេតុ
- បែងចែកធាតុ block, inline និង inline-block
- បង្កើតក្បាលទំព័រ របារនាវីហ្គេសិន និងជួរកាតដោយ Flexbox

6.1 Every element is a box

ធាតុនីមួយៗជាប្រអប់

CSS lays out a page as boxes. Each box has four layers, from the inside out: the **content** (text or image), the **padding** (space inside the border), the **border**, and the **margin** (space outside the border, between this box and its neighbours). Padding takes the element's background colour; margin is always transparent. Open DevTools, select any element and look at the box diagram in the *Computed* panel to see the four layers with their sizes.

By default, `width` sets only the content. Padding and border are added on top, so a box with `width: 200px`, 20 px padding and a 5 px border is really 250 px wide. With `box-sizing: border-box`, the width includes padding and border, which is much easier to reason about. Many developers put this reset at the top of every stylesheet:

```
*, *::before, *::after { box-sizing: border-box; }
```

CSS STYLE.CSS LISTING 6.1 · EX/S6_BOX

```css
.box {
  width: 200px;
  padding: 20px;
  border: 5px solid #1d4ed8;
  margin: 12px 0;
  background: #dbeafe;
}
.content-box { box-sizing: content-box; } /* total width 250px */
.border-box  { box-sizing: border-box;  } /* total width 200px */
```

RESULT IN THE BROWSER · លទ្ធផល CHROMIUM



560px wide

CSS រៀបចំទំព័រជាប្រអប់។ ប្រអប់នីមួយៗមានបួនស្រទាប់ ពីខាងក្នុងទៅក្រៅ៖ **content** (អត្ថបទ ឬរូបភាព) **padding** (ចន្លោះខាងក្នុងស៊ុម) **border** (ស៊ុម) និង **margin** (ចន្លោះខាងក្រៅស៊ុម)។ Padding យកពណ៌ផ្ទៃខាងក្រោយរបស់ធាតុ ចំណែក margin តែងតែថ្លា។ តាមលំនាំដើម `width` កំណត់តែ content ប៉ុណ្ណោះ ដូច្នេះ ប្រអប់ width 200px ដែលមាន padding 20px និង border 5px ពិតជាមានទទឹង 250px។ ជាមួយ `box-sizing: border-box` ទទឹងរួមបញ្ចូល padding និង border ដែលងាយគណនាជាង។

6.2 Block, inline and inline-block

Block, Inline និង Inline-block

Block elements, such as `<p>`, `<h1>`, `<div>` and `<section>`, start on a new line and stretch to the full width available. **Inline** elements, such as `<a>`, `<strong>` and `<span>`, sit inside a line of text, and their width and height cannot be set. **Inline-block** elements flow in a line like text but accept width, height and vertical padding, which is useful for buttons. You can change any element with the `display` property.

One surprise: the vertical margins of two block elements next to each other can **collapse** into one. A 20 px bottom margin followed by a 30 px top margin gives 30 px of space, not 50 px. Flexbox and Grid items do not collapse margins, which is one reason modern layouts use `gap` instead.

ធាតុ **block** ចាប់ផ្តើមលើបន្ទាត់ថ្មី ហើយលាតពេញទទឹង។ ធាតុ **inline** នៅក្នុងបន្ទាត់អត្ថបទ ហើយមិនអាចកំណត់ទទឹង ឬកម្ពស់បានទេ។ ធាតុ **inline-block** ហូរដូចអត្ថបទ ប៉ុន្តែទទួល width, height និង padding បញ្ឈរ។ អ្នកអាចប្តូរដោយ property `display` ។ Margin បញ្ឈររបស់ block ពីរក្បែរគ្នាអាច **រួមបញ្ចូលគ្នា (collapse)**៖ margin 20px និង 30px ផ្តល់ចន្លោះ 30px មិនមែន 50px ទេ។ ធាតុក្នុង Flexbox និង Grid មិនមានបាតុភូតនេះទេ។

6.3 Flexbox: layout in one direction

Flexbox៖ ប្លង់ក្នុងទិសដៅមួយ

Flexbox arranges items in one direction, a row or a column. Set `display: flex` on the **container**; its direct children become **flex items**. The container controls the layout: `flex-direction` chooses row or column, `justify-content` spreads items along the main axis (`flex-start`, `center`, `space-between`), `align-items` aligns them on the cross axis (`stretch`, `center`), `gap` adds space between items, and `flex-wrap: wrap` lets items move to a new line.

CSS STYLE.CSS LISTING 6.2 · EX/S6_FLEXNAV

```css
.site-header {
  display: flex;
  justify-content: space-between;   /* logo left, nav right */
  align-items: center;
  padding: 12px 20px;
  background: #1e293b;
}
.logo { color: #fbbf24; font-weight: bold; text-decoration: none; }
.nav  { display: flex; gap: 16px; list-style: none;
        margin: 0; padding: 0; }
.nav a { color: #f1f5f9; text-decoration: none; }
```

RESULT IN THE BROWSER · លទ្ធផល CHROMIUM



620px wide

Three lines of Flexbox put the logo on the left and the menu on the right, vertically centred. Notice the list of links inside the `<nav>` is itself a flex container, so you can nest flex layouts.

Flexbox រៀបចំធាតុក្នុងទិសដៅមួយ ជាជួរដេក ឬជួរឈរ។ ដាក់ `display: flex` លើ **container** ហើយធាតុកូនផ្ទាល់ក្លាយជា **flex item**។ Container គ្រប់គ្រងប្លង់៖ `flex-direction` ជ្រើសជួរដេក ឬជួរឈរ `justify-content` ចែកចាយធាតុតាមអ័ក្សមេ `align-items` តម្រឹមតាមអ័ក្សទទឹង `gap` បន្ថែមចន្លោះ ហើយ

`flex-wrap: wrap` អនុញ្ញាតឱ្យធាតុចុះបន្ទាត់ថ្មី។ ក្នុង Listing 6.2 Flexbox បីបន្ទាត់ដាក់ឡូហ្គោនៅឆ្វេង និងម៉ឺនុយនៅស្តាំ។

Growing and shrinking items · ការពង្រីក និងបង្រួមធាតុ

The `flex` shorthand on an item sets three values: **grow**, **shrink** and **basis**. `flex: 1 1 160px` means “start at 160 px, grow to share spare space, and shrink if needed”. With `flex-wrap: wrap`, the cards in Listing 6.3 fill the row on a wide screen and wrap onto new lines on a narrow one, without a single media query.

CSS STYLE.CSS LISTING 6.3 · EX/S6_FLEXCARDS

```css
.cards {
  display: flex;
  flex-wrap: wrap;          /* move to a new line when needed */
  gap: 12px;
}
.card {
  flex: 1 1 160px;          /* grow, shrink, start at 160px */
  padding: 12px;
  border: 1px solid #cbd5e1;
  border-radius: 8px;
}
.card h3 { margin: 0 0 4px; }
```

RESULT IN THE BROWSER · លទ្ធផល CHROMIUM



640px wide

Shorthand `flex` លើធាតុកំណត់តម្លៃបី៖ grow, shrink និង basis។ `flex: 1 1 160px` មានន័យថា «ចាប់ផ្តើម 160px ពង្រីកដើម្បីចែករំលែកចន្លោះទំនេរ និងបង្រួមបើចាំបាច់»។ ជាមួយ flex-wrap កាតក្នុង Listing 6.3 ពេញជួរលើអេក្រង់ធំ ហើយចុះបន្ទាត់លើអេក្រង់តូច ដោយមិនចាំបាច់មាន media query។

KEY IDEA គំនិតសំខាន់

Use **Flexbox** for one-dimensional layout: a row of buttons, a navigation bar, a card’s content stacked in a column. Use **Grid** (Session 7) when you need rows and columns at the same time.

ប្រើ Flexbox សម្រាប់ប្លង់មួយវិមាត្រ៖ ជួរប៊ូតុង របារនាវីហ្គេសិន។ ប្រើ Grid (វគ្គ ៧) ពេលត្រូវការទាំងជួរដេក និងជួរឈរក្នុងពេលតែមួយ។

IN THE AI ERA ក្នុងយុគសម័យ AI

Layout bugs are hard to describe in words, and AI tools now accept screenshots. Send a screenshot of the problem together with the HTML and CSS of that part, and ask: “Explain why the items do not line up. Show the smallest change.” Then test the change in DevTools before editing your file. If the suggestion uses fixed pixel widths or negative margins to force things into place, ask for a Flexbox-based alternative.

បញ្ហាប្លង់ពិបាកពិពណ៌នាជាពាក្យ ហើយឧបករណ៍ AI ឥឡូវទទួលរូបថតអេក្រង់។ ផ្ញើរូបថតអេក្រង់ ជាមួយ HTML និង CSS នៃផ្នែកនោះ ហើយសួរ «ពន្យល់ហេតុអ្វីធាតុមិនតម្រឹម។ បង្ហាញការផ្លាស់ប្តូរតូចបំផុត»។ សាកល្បងក្នុង DevTools ជាមុនសិន។

LAB 6 · HEADER, NAVIGATION AND CARDS Lab ៦ · ក្បាលទំព័រ នាវីហ្គេសិន និងកាត

1. Add the `box-sizing` reset to the top of your stylesheet.
   បន្ថែម box-sizing reset នៅខាងលើឯកសារ CSS។
2. Turn your site header into a flex container with your name on the left and the navigation on the right.
   ធ្វើឱ្យក្បាលទំព័រជា flex container។
3. On `projects.html`, show three projects as cards in a wrapping flex row, each with a title, a short description and a link.
   បង្ហាញគម្រោងបីជាកាតក្នុងជួរ flex។
4. Use DevTools’ device toolbar to view the page at 360 px wide. Fix anything that overflows.
   មើលទំព័រនៅទទឹង 360px ហើយកែអ្វីដែលហៀរចេញ។
5. **Mini project (20%):** hand in your three-page site, styled and validated.
   គម្រោងតូច៖ ប្រគល់គេហទំព័របីទំព័រ ដែលបានរចនា និងផ្ទៀងផ្ទាត់។

Exercises

លំហាត់

1. A box has `width: 300px; padding: 10px; border: 2px solid; margin: 20px;`. How wide is it with `content-box` and with `border-box` (not counting margin)?
   គណនាទទឹងប្រអប់ក្នុងករណីទាំងពីរ។
2. Why can’t you set the width of a `<span>`? How can you make it possible?
   ហេតុអ្វីមិនអាចកំណត់ទទឹង span? ធ្វើដូចម្តេចឱ្យអាច?
3. Centre a single box horizontally and vertically inside a container using Flexbox.

ដាក់ប្រអប់មួយកណ្តាលទាំងផ្តេក និងបញ្ឈរដោយ Flexbox។

4. What is the difference between `justify-content` and `align-items` when `flex-direction` is `column`?
   តើ justify-content និង align-items ខុសគ្នាដូចម្តេចពេលជាជួរឈរ?
5. Explain `flex: 2 1 0` compared with `flex: 1 1 0` for two items in one row.
   ពន្យល់ flex: 2 1 0 ធៀបនឹង flex: 1 1 0។

SESSION 7

CSS GRID AND RESPONSIVE DESIGN

CSS Grid និងការរចនាឆ្លើយតបតាមអេក្រង់

IN THIS SESSION ក្នុងវគ្គនេះ

- Build two-dimensional layouts with CSS Grid and named areas
- Design mobile-first and add breakpoints with media queries
- Make grids that adapt without media queries
- Serve responsive images that do not waste mobile data
- បង្កើតប្លង់ពីរវិមាត្រដោយ CSS Grid និងតំបន់មានឈ្មោះ
- រចនាពីទូរស័ព្ទជាមុន (mobile-first) និងបន្ថែម breakpoint ដោយ media query
- បង្កើត grid ដែលសម្របខ្លួនដោយគ្មាន media query
- ផ្តល់រូបភាពឆ្លើយតប ដែលមិនខ្ជះខ្ជាយទិន្នន័យទូរស័ព្ទ

7.1 CSS Grid basics

មូលដ្ឋាន CSS Grid

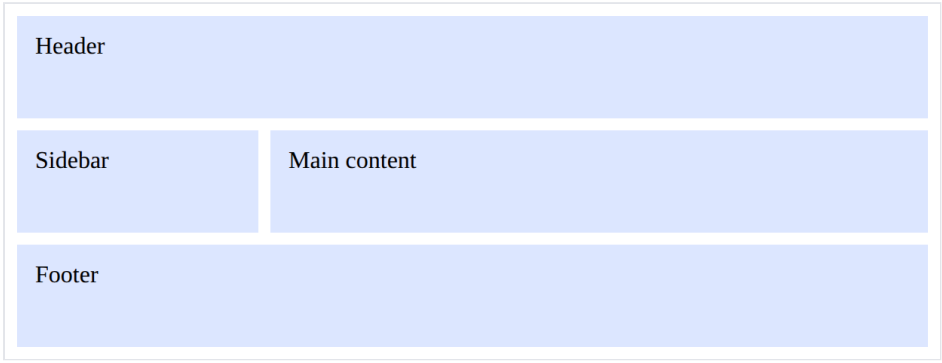
CSS Grid lays out items in rows and columns at the same time. Set `display: grid` on the container and describe the columns with `grid-template-columns`. The `fr` unit means “a fraction of the free space”, so `1fr 2fr` makes the second column twice as wide as the first, and `repeat(3, 1fr)` makes three equal columns. `gap` sets the space between rows and columns.

For page layouts, **named areas** are the clearest tool. You draw the layout as text in `grid-template-areas` and then assign each element to an area with `grid-area`. The CSS almost looks like the wireframe you sketched in Session 4.

CSS STYLE.CSS LISTING 7.1 · EX/S7_GRID

```css
.page {
  display: grid;
  grid-template-columns: 160px 1fr;
  grid-template-areas:
    "header header"
    "nav    main"
    "footer footer";
  gap: 8px;
  min-height: 220px;
}
header { grid-area: header; }
nav    { grid-area: nav; }
main   { grid-area: main; }
footer { grid-area: footer; }
.page > * { background: #e0e7ff; padding: 12px; }
```

RESULT IN THE BROWSER · លទ្ធផល CHROMIUM



620px wide

CSS Grid រៀបចំធាតុជាជួរដេក និងជួរឈរក្នុងពេលតែមួយ។ ដាក់ `display: grid` លើ container ហើយពិពណ៌នាជួរឈរដោយ `grid-template-columns` ។ ឯកតា `fr` មានន័យថា «ចំណែកនៃចន្លោះទំនេរ» ដូច្នេះ `1fr 2fr` ធ្វើឱ្យជួរឈរទីពីរធំជាងទីមួយពីរដង ហើយ `repeat(3, 1fr)` បង្កើតជួរឈរស្មើគ្នាបី។ សម្រាប់ប្លង់ទំព័រ **តំបន់មានឈ្មោះ (named areas)** ជាឧបករណ៍ច្បាស់បំផុត៖ អ្នកគូរប្លង់ជាអត្ថបទក្នុង `grid-template-areas` រួចកំណត់ធាតុនីមួយៗទៅតំបន់ដោយ `grid-area` ។

7.2 Responsive, mobile-first design

ការរចនាឆ្លើយតប ចាប់ពីទូរសព្ទ

Responsive web design, a term introduced by Ethan Marcotte in 2010, means one page that adapts to any screen with flexible layouts, flexible images and media queries [14]. In Cambodia, as in much of the world, many visitors browse on phones, so design for the small screen first. **Mobile-first** CSS writes the simple one-column layout as the default,

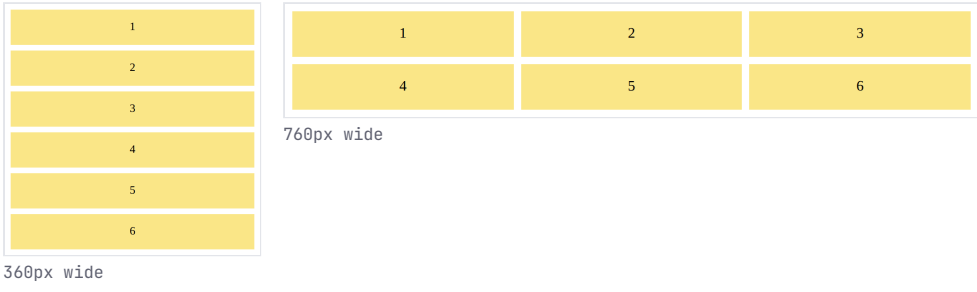
then adds rules for wider screens with `@media (min-width: …)`. The viewport meta tag from Session 1 is required; without it, phones pretend to be a wide desktop screen and shrink the page.

CSS STYLE.CSS LISTING 7.2 · EX/S7_RESPONSIVE

```css
/* mobile first: one column */
.gallery {
  display: grid;
  grid-template-columns: 1fr;
  gap: 8px;
}
.gallery div { background: #fde68a; padding: 16px; text-align: center; }

/* tablet and up: three columns */
@media (min-width: 600px) {
  .gallery { grid-template-columns: repeat(3, 1fr); }
}
```

RESULT IN THE BROWSER · លទ្ធផល CHROMIUM



Choose **breakpoints** where your content starts to look wrong, not where a particular phone model ends. Test by dragging the browser window narrower and wider, and with the device toolbar in DevTools.

> **ការរចនាគេហទំព័រឆ្លើយតប (responsive)** ជាពាក្យដែល Ethan Marcotte បានណែនាំក្នុងឆ្នាំ ២០១០ មានន័យថាទំព័រតែមួយសម្របខ្លួនទៅនឹងអេក្រង់គ្រប់ទំហំ ដោយប្លង់ និងរូបភាពបត់បែន និង media query។ នៅកម្ពុជា ដូចកន្លែងជាច្រើនទៀត អ្នកទស្សនាជាច្រើនប្រើទូរស័ព្ទ ដូច្នេះត្រូវរចនាសម្រាប់អេក្រង់តូចជាមុន។ CSS **mobile-first** សរសេរប្លង់មួយជួរឈរជាលំនាំដើម រួចបន្ថែមច្បាប់សម្រាប់អេក្រង់ធំជាមួយ `@media (min-width: …)` ។ ជ្រើស breakpoint នៅកន្លែងដែលមាតិកាចាប់ផ្តើមមើលទៅខុស មិនមែនតាមម៉ូដែលទូរស័ព្ទណាមួយទេ។

7.3 Grids that adapt by themselves

Grid ដែលសម្របខ្លួនដោយខ្លួនឯង

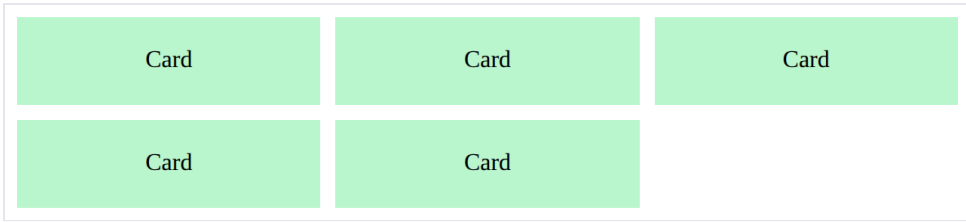
Often you do not need a media query at all. `repeat(auto-fit, minmax(150px, 1fr))` tells the browser: “make as many columns as fit, each at least 150 px wide, and share the

leftover space”. On a phone you get one column, on a laptop four or five. This is called **intrinsic** design: the layout responds to the space it has.

CSS STYLE.CSS LISTING 7.3 · EX/S7_AUTOFIT

```css
/* no media query needed: columns appear when there is space */
.grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(150px, 1fr));
  gap: 10px;
}
.grid div { background: #bbf7d0; padding: 20px; text-align: center; }
```

RESULT IN THE BROWSER · លទ្ធផល CHROMIUM



640px wide

Modern CSS also offers **container queries**, which let a component change its layout based on the width of its own container instead of the whole screen. They are supported in all major browsers; check the Baseline label on MDN before you rely on any newer feature [15].

ជារឿយៗអ្នកមិនត្រូវការ media query ទាល់តែសោះ។ `repeat(auto-fit, minmax(150px, 1fr))` ប្រាប់ browser ថា «បង្កើតជួរឈរឱ្យបានច្រើនតាមដែលអាចដាក់បាន ជួរនីមួយៗយ៉ាងតិច 150px ហើយចែករំលែកចន្លោះដែលនៅសល់»។ លើទូរស័ព្ទអ្នកបានមួយជួរឈរ លើកុំព្យូទ័របួន ឬប្រាំ។ CSS ទំនើបក៏មាន **container query** ដែលអនុញ្ញាតឱ្យសមាសធាតុប្តូរប្លង់តាមទទឹងរបស់ container ខ្លួនឯង។ សូមពិនិត្យស្លាក Baseline លើ MDN មុនពេលប្រើមុខងារថ្មីៗ។

DEFINITION និយមន័យ

Baseline is a label used by MDN and web.dev. A feature is “newly available” when it works in all the core browsers (Chrome, Edge, Firefox and Safari, on desktop and mobile), and “widely available” 30 months after that [15].

Baseline ជាស្លាកដែល MDN និង web.dev ប្រើ។ មុខងារមួយ «ទើបមាន» ពេលវាដំណើរការក្នុង browser ស្នូលទាំងអស់ ហើយ «មានទូលំទូលាយ» ៣០ ខែក្រោយមក។

7.4 Responsive images

រូបភាពឆ្លើយតប

A simple rule stops images from breaking narrow layouts: `img { max-width: 100%; height: auto; }`. To save mobile data, offer several sizes and let the browser pick the smallest one that looks sharp:

```
<img src="team-800.jpg" srcset="team-400.jpg 400w, team-800.jpg 800w, team-1600.jpg 1600w" sizes="(min-width: 800px) 50vw, 100vw" alt="Our team at the café" width="800" height="533">
```

`srcset` lists the files and their real widths, and `sizes` tells the browser how wide the image will appear. Add `loading="lazy"` to images lower down the page so they load only when the user scrolls near them.

ច្បាប់សាមញ្ញមួយការពាររូបភាពកុំឱ្យបំបែកប្លង់តូច៖ `img { max-width: 100%; height: auto; }`។ ដើម្បីសន្សំទិន្នន័យទូរស័ព្ទ ផ្តល់រូបភាពច្រើនទំហំ ហើយឱ្យ browser ជ្រើសទំហំតូចបំផុតដែលនៅតែច្បាស់។ `srcset` រាយឯកសារ និងទទឹងពិត ហើយ `sizes` ប្រាប់ថារូបភាពនឹងបង្ហាញធំប៉ុណ្ណា។ បន្ថែម `loading="lazy"` សម្រាប់រូបភាពនៅខាងក្រោមទំព័រ។

IN THE AI ERA ក្នុងយុគសម័យ AI

Give an AI assistant your wireframe sketch and ask it to convert the layout to Grid with named areas, “mobile-first, one breakpoint, no fixed heights”. Constraints like these lead to cleaner code. Then test at 320, 375, 768 and 1280 px. AI-generated layouts often look fine at one width and overflow at another, because the tool did not see your page at every size.

ផ្តល់គំនូរប្លង់ព្រាងរបស់អ្នកទៅ AI ហើយសុំឱ្យបំប្លែងជា Grid ជាមួយ named areas «mobile-first, breakpoint មួយ, គ្មានកម្ពស់ថេរ»។ រួចសាកល្បងនៅទទឹង 320, 375, 768 និង 1280px ព្រោះប្លង់ពី AI ជារឿយៗល្អនៅទទឹងមួយ តែហៀរនៅទទឹងផ្សេង។

LAB 7 · A MOBILE-FIRST PORTFOLIO Lab ៧ · ផលប័ត្រ mobile-first

1. Rebuild your home page layout with Grid and named areas: header, main, aside (for example “Now learning”) and footer.
   បង្កើតប្លង់ទំព័រដើមឡើងវិញដោយ Grid និង named areas។
2. Write it mobile-first: one column by default, two columns from a breakpoint you choose by testing.
   សរសេរ mobile-first៖ មួយជួរឈរជាលំនាំដើម ពីរជួរឈរចាប់ពី breakpoint មួយ។
3. Change the project cards to an `auto-fit` grid.

ប្តូរកាតគម្រោងទៅជា grid auto-fit។

4. Make all images responsive, and add `loading="lazy"` where it makes sense.
   ធ្វើឱ្យរូបភាពទាំងអស់ឆ្លើយតប។
5. Test on a real phone. Record two problems you did not see in DevTools.
   សាកល្បងលើទូរស័ព្ទពិត ហើយកត់ត្រាបញ្ហាពីរ។

Exercises

លំហាត់

1. Write `grid-template-columns` for a sidebar of 250 px and a main column that takes the rest.
   សរសេរ grid-template-columns សម្រាប់ sidebar 250px និង main។
2. Rewrite this desktop-first query mobile-first: `@media (max-width: 599px) { .grid { grid-template-columns: 1fr; } }`.
   សរសេរ media query នេះឡើងវិញជាបែប mobile-first។
3. What happens to `repeat(auto-fit, minmax(200px, 1fr))` in a container 650 px wide with a 10 px gap? How many columns?
   តើមានជួរឈរប៉ុន្មាន?
4. When would you choose Flexbox instead of Grid? Give two examples.
   ពេលណាជ្រើស Flexbox ជំនួស Grid?
5. Explain what `sizes="(min-width: 800px) 50vw, 100vw"` tells the browser.
   ពន្យល់ពីអ្វីដែល sizes ប្រាប់ browser។

SESSION 8

MODERN CSS: VARIABLES, STATES AND DARK MODE

CSS ទំនើប៖ អថេរ ស្ថានភាព និងរបៀបងងឹត

IN THIS SESSION ក្នុងវគ្គនេះ

- Store design decisions in CSS custom properties (design tokens)
- Use `clamp()` for sizes that grow with the screen
- Support dark mode and reduced motion from user settings
- Style hover, focus and disabled states, with smooth transitions
- Think in reusable components

- រក្សាទុកការសម្រេចចិត្តរចនាក្នុង CSS custom property (design token)
- ប្រើ clamp() សម្រាប់ទំហំដែលកើនតាមអេក្រង់
- គាំទ្ររបៀបងងឹត និងការកាត់បន្ថយចលនា តាមការកំណត់អ្នកប្រើ
- រចនាស្ថានភាព hover, focus និង disabled ជាមួយ transition
- គិតជាសមាសធាតុដែលប្រើឡើងវិញបាន

8.1 Custom properties as design tokens

Custom property ជា design token

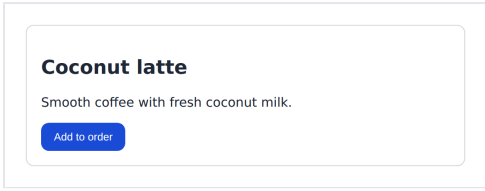
A **custom property**, often called a CSS variable, stores a value under a name that starts with two dashes: `--brand: #1d4ed8;`. Read it with `var(--brand)`. Define global values on `:root` so the whole page can use them. Teams call these named values **design tokens**: the colours, spacing, radius and font sizes of the style guide, written once and reused everywhere. Change one token and the whole site updates.

The `clamp(min, preferred, max)` function keeps a value between two limits. `clamp(12px, 3vw, 24px)` is 3% of the viewport width, but never smaller than 12 px or larger than 24 px, so spacing and headings grow smoothly with the screen without extra media queries.

CSS STYLE.CSS LISTING 8.1 · EX/S8_TOKENS

```css
:root {
  --bg: #ffffff;   --text: #1f2937;
  --brand: #1d4ed8; --radius: 10px;
  --space: clamp(12px, 3vw, 24px);   /* grows with the screen */
}
@media (prefers-color-scheme: dark) {
  :root { --bg: #111827; --text: #f3f4f6; --brand: #60a5fa; }
}
body  { background: var(--bg); color: var(--text);
        font-family: system-ui, sans-serif; padding: var(--space); }
.card { border: 1px solid color-mix(in srgb, var(--text) 20%, transparent);
        border-radius: var(--radius); padding: var(--space); }
.btn  { background: var(--brand); color: var(--bg); border: 0;
        padding: 10px 16px; border-radius: var(--radius); }
```

RESULT IN THE BROWSER · លទ្ធផល CHROMIUM



600px · light wide



600px · dark wide

Custom property (ជារឿយៗហៅថាអថេរ CSS) រក្សាទុកតម្លៃក្រោមឈ្មោះដែលចាប់ផ្តើមដោយសញ្ញាដកពីរ៖ `--brand: #1d4ed8;` ។ អានវាដោយ `var(--brand)` ។ កំណត់តម្លៃសកលលើ `:root` ។ ក្រុមការងារហៅតម្លៃមានឈ្មោះទាំងនេះថា **design token**៖ ពណ៌ គម្លាត កាំជ្រុង និងទំហំអក្សរនៃមគ្គុទ្ទេសក៍រចនាប័ទ្ម ដែលសរសេរម្តង ហើយប្រើគ្រប់ទីកន្លែង។ ប្តូរ token មួយ គេហាទំព័រទាំងមូលប្តូរតាម។ អនុគមន៍ `clamp(min, preferred, max)` រក្សាតម្លៃនៅចន្លោះដែនកំណត់ពីរ។

8.2 Respecting user settings

ការគោរពការកំណត់របស់អ្នកប្រើ

Operating systems let people choose a dark appearance and reduce animations. CSS can read these choices with media features. `@media (prefers-color-scheme: dark)` applies when the user prefers dark mode; in Listing 8.1 it only redefines the colour tokens, and every component follows automatically. The two screenshots show the same page with the system set to light and to dark. Check contrast again for the dark theme: a colour that passes on white may fail on near-black.

`@media (prefers-reduced-motion: reduce)` applies when the user has asked for less motion, for example because animation makes them dizzy. Turn off or shorten non-essential animations there.

ប្រព័ន្ធប្រតិបត្តិការអនុញ្ញាតឱ្យមនុស្សជ្រើសរូបរាងងងឹត និងកាត់បន្ថយចលនា។ CSS អាចអានជម្រើសទាំងនេះ។ `@media (prefers-color-scheme: dark)` អនុវត្តពេលអ្នកប្រើចូលចិត្តរបៀបងងឹត។ ក្នុង Listing 8.1 វាគ្រាន់តែកំណត់ token ពណ៌ឡើងវិញ ហើយសមាសធាតុទាំងអស់ធ្វើតាមដោយស្វ័យប្រវត្តិ។ សូមពិនិត្យកម្រិតពណ៌ផ្ទុយម្តងទៀតសម្រាប់ទម្រង់ងងឹត។ `@media (prefers-reduced-motion: reduce)` អនុវត្តពេលអ្នកប្រើស្នើចលនាតិច ដូច្នេះសូមបិទចលនាដែលមិនចាំបាច់។

8.3 Interactive states and transitions

ស្ថានភាពអន្តរកម្ម និង Transition

Buttons and links need clear states. `:hover` applies when a mouse pointer is over the element, but touch screens have no hover, so never hide important information behind it. `:focus-visible` applies when the element has keyboard focus; never remove the focus outline without giving a clear replacement, because keyboard users would lose their place [6]. `:disabled` styles controls that cannot be used.

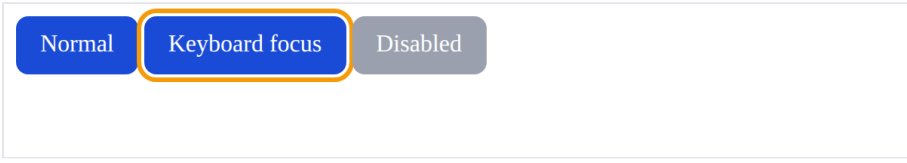
A **transition** animates a change between two states: `transition: background-color .2s ease;`. Keep interface transitions short, roughly 150 to 300 milliseconds, and animate `transform` and `opacity` where you can, because browsers can animate them smoothly.

CSS STYLE.CSS LISTING 8.2 · EX/S8_STATES

```css
.btn {
  font: inherit; padding: 10px 16px; border: 0; border-radius: 8px;
  background: #1d4ed8; color: #fff; cursor: pointer;
  transition: background-color .2s ease, transform .2s ease;
}
.btn:hover         { background: #1e40af; transform: translateY(-1px); }
.btn:focus-visible { outline: 3px solid #f59e0b; outline-offset: 2px; }
.btn:disabled      { background: #9ca3af; cursor: not-allowed; }

@media (prefers-reduced-motion: reduce) {
  .btn { transition: none; }
}
```

RESULT IN THE BROWSER · លទ្ធផល CHROMIUM



600px wide

In the screenshot, the middle button has keyboard focus, so it shows the amber outline. The disabled button is grey and shows a “not allowed” cursor when a mouse is over it.

ប៊ូតុង និងតំណត្រូវការស្ថានភាពច្បាស់។ `:hover` អនុវត្តពេលព្រួញកណ្តុរនៅលើធាតុ ប៉ុន្តែអេក្រង់ប៉ះគ្មាន hover ទេ ដូច្នេះកុំលាក់ព័ត៌មានសំខាន់ពីក្រោយវា។ `:focus-visible` អនុវត្តពេលធាតុមានការផ្តោតពីក្តារចុច។ កុំលុបបន្ទាត់ផ្តោតដោយគ្មានជំនួសច្បាស់លាស់។ `:disabled` រចនាធាតុដែលមិនអាចប្រើបាន។ **Transition** បង្ហាញចលនាពេលប្តូរស្ថានភាព។ រក្សាវាឱ្យខ្លី ប្រហែល 150 ដល់ 300 មីលីវិនាទី ហើយប្រើ transform និង opacity។

8.4 Thinking in components

ការគិតជាសមាសធាតុ

Professional sites are built from reusable **components**: a button, a card, a navigation bar, a form field. Brad Frost's *Atomic Design* describes this as building small "atoms" (a label, a button) into "molecules" (a search form) and "organisms" (a header) [16]. In CSS, give each component one clear class name, such as `.card`, and name its parts and variants consistently, for example `.card__title` and `.card--featured` (a convention called BEM). A component should look right wherever you put it.

គេហទំព័រអាជីពត្រូវបានសាងសង់ពី **សមាសធាតុ (component)** ដែលប្រើឡើងវិញបាន៖ ប៊ូតុង កាត របារនាវីហ្គេសិន វាលទម្រង់។ *Atomic Design* របស់ Brad Frost ពិពណ៌នាការធ្មុំ «អាតូម» តូចៗទៅជា «ម៉ូលេគុល» និង «សរីរាង្គ»។ ក្នុង CSS ផ្តល់ឈ្មោះ class ច្បាស់មួយសម្រាប់សមាសធាតុនីមួយៗ ដូចជា .card ហើយដាក់ឈ្មោះផ្នែក និងបំរែបំរួលឱ្យស៊ីសង្វាក់ ឧទាហរណ៍ .card__title និង .card--featured (ហៅថា BEM)។

IN THE AI ERA ក្នុងយុគសម័យ AI

AI assistants are good at refactoring repetitive CSS into tokens. Try: "Find every colour and spacing value in my stylesheet and turn them into custom properties on `:root`. Keep the visual result identical." Then compare screenshots before and after. Ask it to add a dark theme only by redefining tokens, and check every text colour against its background with a contrast checker: AI-made dark themes often have low-contrast grey text.

AI ពូកែកែសម្រួល CSS ដែលធ្វើឡើងវិញច្រើនដង ទៅជា token។ សុំឱ្យវារកពណ៌ និងគម្លាតទាំងអស់ ហើយបំប្លែងជា custom property ដោយរក្សារូបរាងដដែល។ ប្រៀបធៀបរូបថតអេក្រង់មុន និងក្រោយ។ ពិនិត្យកម្រិតពណ៌ផ្ទុយនៃទម្រង់ងងឹតដែល AI បង្កើត ព្រោះវាតែងមានអក្សរប្រផេះកម្រិតផ្ទុយទាប។

LAB 8 · TOKENS AND A DARK THEME Lab ៨ · Token និងទម្រង់ងងឹត

1. Move every colour, font, radius and spacing value in your stylesheet into tokens on `:root`.
   ផ្លាស់ពណ៌ ពុម្ពអក្សរ កាំជ្រុង និងគម្លាតទាំងអស់ទៅជា token។

2. Add a dark theme with `prefers-color-scheme`. All text must still reach 4.5:1 contrast.
   បន្ថែមទម្រង់ងងឹត ហើយអត្ថបទទាំងអស់ត្រូវតែមានកម្រិតផ្ទុយ 4.5:1។
3. Create a `.btn` component with hover, focus-visible and disabled states, and use it for every button and call-to-action link.
   បង្កើតសមាសធាតុ .btn ជាមួយស្ថានភាពទាំងបី។
4. Use `clamp()` for your main heading size.
   ប្រើ clamp() សម្រាប់ទំហំចំណងជើងចម្បង។
5. Turn on “reduce motion” in your operating system and check that your transitions stop.
   បើក «reduce motion» ហើយពិនិត្យថា transition ឈប់។

Exercises

លំហាត់

1. What does `var(--gap, 16px)` mean? (Look up the second argument on MDN.)
   តើ var(--gap, 16px) មានន័យដូចម្តេច?
2. At what viewport width does `clamp(1rem, 4vw, 2rem)` stop growing, if 1rem = 16 px?
   តើ clamp នេះឈប់កើននៅទទឹងប៉ុន្មាន?
3. Why is `outline: none` on `:focus` an accessibility problem?
   ហេតុអ្វី outline: none ជាបញ្ហា?
4. Write a transition that fades a card in by animating opacity over 200 ms.
   សរសេរ transition ដែលបង្ហាញកាតដោយ opacity ក្នុង 200ms។
5. Name the BEM classes for a card with an image, a title and a “sale” variant.
   ដាក់ឈ្មោះ class BEM សម្រាប់កាតនេះ។

SESSION 9

JAVASCRIPT FOUNDATIONS

មូលដ្ឋាន JavaScript

IN THIS SESSION ក្នុងវគ្គនេះ

- Add JavaScript to a page and use the console
- Store values with `const` and `let`, and know the main data types
- Make decisions with `if` and repeat work with loops
- Write functions, and process arrays of objects with `map`, `filter` and `reduce`
- បន្ថែម JavaScript ទៅទំព័រ និងប្រើ console
- រក្សាទុកតម្លៃដោយ const និង let ហើយស្គាល់ប្រភេទទិន្នន័យចម្បង
- ធ្វើការសម្រេចចិត្តដោយ if និងធ្វើការងារឡើងវិញដោយរង្វិលជុំ
- សរសេរអនុគមន៍ និងដំណើរការអារេនៃវត្ថុដោយ map, filter និង reduce

9.1 Where JavaScript runs

តើ JavaScript ដំណើរការនៅឯណា

JavaScript is the programming language of the browser. Its official specification is called ECMAScript [17]. Put your code in a separate file and load it at the end of `<body>`, or in the head with `defer`, so the HTML exists before the script runs:

```
<script src="script.js" defer></script>
```

Your best friend is `console.log()`, which prints values to the *Console* panel of DevTools. Errors appear there too, in red, with a file name and a line number. Read them: they usually tell you exactly what is wrong. JavaScript also runs outside the browser in Node.js; the console outputs in this session were produced with Node.js 22.

> JavaScript ជាភាសាសរសេរកម្មវិធីរបស់ browser ហើយលក្ខណៈបច្ចេកទេសផ្លូវការរបស់វាហៅថា ECMAScript។ ដាក់កូដក្នុងឯកសារដាច់ដោយឡែក ហើយផ្ទុកវានៅចុង body ឬក្នុង head ជាមួយ `defer` ដើម្បីឱ្យ HTML មានរួចមុនពេលស្ក្រីបដំណើរការ។ `console.log()` បោះពុម្ពតម្លៃទៅ Console ក្នុង DevTools។ កំហុសក៏បង្ហាញនៅទីនោះជាពណ៌ក្រហម ជាមួយឈ្មោះឯកសារ និងលេខបន្ទាត់។ សូមអានវា ព្រោះវាតែងប្រាប់អ្នកច្បាស់ពីអ្វីដែលខុស។

9.2 Values, variables and types

តម្លៃ អថេរ និងប្រភេទ

Declare a variable with `const` when the name will always point to the same value, and with `let` when it will change. Avoid the old `var`. The main **primitive types** are **string** (text in quotes), **number** (integers and decimals together), **boolean** (`true` or `false`), **undefined** (no value yet) and **null** (intentionally empty). Arrays and objects, covered below, are not primitives. **Template literals** in back-ticks put values inside text: `` `Total: $${total}` ``.

JS SCRIPT.JS LISTING 9.1 · EX/S9_BASICS

```
const shop = "Mekong Coffee";   // cannot be reassigned
let cups = 3;                     // can change
cups = cups + 2;

const price = 1.5;
const total = cups * price;

console.log(shop, "sold", cups, "cups");
console.log(`Total: $${total.toFixed(2)}`);
console.log(typeof shop, typeof cups, typeof true);
console.log("5" + 2, "5" * 2);   // careful with types!
console.log(0.1 + 0.2 === 0.3);
```

CONSOLE OUTPUT · លទ្ធផលក្នុង CONSOLE

```
Mekong Coffee sold 5 cups
Total: $7.50
string number boolean
52 10
false
```

Look at the last two lines. `"5" + 2` gives `"52"` because `+` joins text when one side is a string, while `"5" * 2` gives `10` because `*` converts the string to a number. And `0.1 + 0.2 === 0.3` is `false` because computers store decimals in binary with tiny rounding errors. For money, round for display with `toFixed(2)`, or count in whole cents or riel.

ប្រកាសអថេរដោយ `const` ពេលឈ្មោះតែងចង្អុលទៅតម្លៃដដែល និង `let` ពេលវានឹងប្តូរ។ ជៀសវាង `var` ចាស់។ ប្រភេទ **primitive** ចម្បងគឺ string (អត្ថបទ) number (ចំនួន) boolean (true/false) undefined (មិនទាន់មានតម្លៃ) និង null (ទទេដោយចេតនា)។ **Template literal** ក្នុងសញ្ញា back-tick ដាក់តម្លៃក្នុងអត្ថបទ។ `"5" + 2` ផ្តល់ `"52"` ព្រោះ + តភ្ជាប់អត្ថបទ ចំណែក `"5" * 2` ផ្តល់ `10` ។ ហើយ `0.1 + 0.2 === 0.3` គឺ false ព្រោះកុំព្យូទ័ររក្សាទុកលេខទសភាគជាគោលពីរ ដែលមានកំហុសបង្គត់តូចៗ។ សម្រាប់ប្រាក់ ចូរបង្គត់ដោយ toFixed(2) ឬរាប់ជាសេន ឬរៀលគត់។

KEY IDEA គំនិតសំខាន់

Always compare with `===` (strict equality), which also checks the type. The loose `==` converts types first, so `"0" == 0` is `true`, which causes surprising bugs.

ប្រៀបធៀបដោយ `===` ជានិច្ច ព្រោះវាពិនិត្យប្រភេទផងដែរ។ `==` បំប្លែងប្រភេទជាមុន ដូច្នេះ `"0" == 0` ជា true ដែលបង្កកំហុសគួរឱ្យភ្ញាក់ផ្អើល។

9.3 Functions, decisions and loops

អនុគមន៍ ការសម្រេចចិត្ត និងរង្វិលជុំ

A **function** is a named, reusable block of code. It takes **parameters** and can `return` a result. Inside, `if`, `else if` and `else` choose what to do; the logical operators `&&` (and), `||` (or) and `!` (not) combine conditions. A `for` loop repeats code a known number of times.

JS SCRIPT.JS LISTING 9.2 · EX/S9_CONTROL

```
function discount(total, isStudent) {
  if (isStudent && total >= 5) {
    return total * 0.9;        // 10% off
  } else if (isStudent) {
    return total * 0.95;       // 5% off
  }
  return total;
}

console.log(discount(10, true));
console.log(discount(4, true));
console.log(discount(10, false));

for (let i = 1; i <= 3; i++) {
  console.log(`Order #${i} is ready`);
}
```

CONSOLE OUTPUT · លទ្ធផលក្នុង CONSOLE

```
9
3.8
10
Order #1 is ready
Order #2 is ready
Order #3 is ready
```

Each call to `discount` runs the same rules with different inputs, which is the whole point of a function. A student spending $10 pays $9; a student spending $4 gets 5% off and pays $3.80; others pay the full price. For much more practice, the free online book *Eloquent JavaScript* is an excellent next step [18].

អនុគមន៍ (function) ជាប្លុកកូដដែលមានឈ្មោះ ហើយប្រើឡើងវិញបាន។ វាទទួល **ប៉ារ៉ាម៉ែត្រ** ហើយអាច `return` លទ្ធផល។ នៅខាងក្នុង `if`, `else if` និង `else` ជ្រើសរើសអ្វីដែលត្រូវធ្វើ។ ប្រមាណវិធីតក្ក `&&` (និង) `||` (ឬ) និង `!` (មិន) បញ្ចូលលក្ខខណ្ឌ។ រង្វិលជុំ `for` ធ្វើកូដឡើងវិញចំនួនដងដែលដឹងជាមុន។ ការហៅ discount នីមួយៗដំណើរការច្បាប់ដដែលជាមួយធាតុចូលខុសគ្នា។

9.4 Arrays and objects

អារេ និងវត្ថុ

An **array** is an ordered list in square brackets; items are numbered from 0. An **object** groups named values, called properties, in curly brackets: `{ name: "Latte", price: 2.25 }`. Real data is usually an array of objects, such as a menu, a list of students or search results.

Three array methods do most of the everyday work, each taking a small function, often written as an **arrow function** `item => …`. `map` makes a new array by transforming each item. `filter` keeps the items that pass a test. `reduce` combines all items into one value, such as a total.

JS SCRIPT.JS LISTING 9.3 · EX/S9_ARRAYS

```
const menu = [
  { name: "Iced coffee",    price: 1.5,  hot: false },
  { name: "Black coffee",   price: 1.25, hot: true  },
  { name: "Coconut latte",  price: 2.25, hot: false },
];

const names = menu.map(item => item.name);
const cold  = menu.filter(item => !item.hot);
const sum   = menu.reduce((acc, item) => acc + item.price, 0);

console.log(names);
console.log(cold.length, "cold drinks");
console.log("Average price:", (sum / menu.length).toFixed(2));
console.log(menu[2].name.toUpperCase());
```

CONSOLE OUTPUT · លទ្ធផលក្នុង CONSOLE

```
[ 'Iced coffee', 'Black coffee', 'Coconut latte' ]
2 cold drinks
Average price: 1.67
COCONUT LATTE
```

អារេ (array) ជាបញ្ជីមានលំដាប់ក្នុងតង្កៀបការ៉េ ធាតុរាប់ពីលេខ 0។ **វត្ថុ (object)** ដាក់តម្លៃមានឈ្មោះ (property) ជាក្រុមក្នុងតង្កៀបរួញ។ ទិន្នន័យពិតជាធម្មតាជាអារេនៃវត្ថុ ដូចជាម៉ឺនុយ ឬបញ្ជីនិស្សិត។ វិធីសាស្ត្របីធ្វើការងារភាគច្រើន៖ `map` បង្កើតអារេថ្មីដោយបំប្លែងធាតុនីមួយៗ `filter` រក្សាធាតុដែលឆ្លងកាត់ការសាកល្បង ហើយ `reduce` បញ្ចូលធាតុទាំងអស់ទៅជាតម្លៃតែមួយ ដូចជាផលបូក។

IN THE AI ERA ក្នុងយុគសម័យ AI

This is the session where AI can most easily stop you from learning. Programming skill grows from struggling a little with each problem. Try each exercise yourself for at least 15 minutes first. When you are stuck, ask for a **hint**, not a solution: "Give me a hint for why my loop prints undefined. Do not write the code." When you get code, change one thing and predict the output before you run it. If you cannot predict it, you do not understand it yet.

វគ្គនេះជាវគ្គដែល AI អាចរារាំងការរៀនរបស់អ្នកបានងាយបំផុត។ ជំនាញសរសេរកម្មវិធីកើនឡើងពីការតស៊ូបន្តិចជាមួយបញ្ហានីមួយៗ។ សាកល្បងលំហាត់ដោយខ្លួនឯងយ៉ាងតិច ១៥ នាទីជាមុន។ ពេលជាប់គាំង សុំ **ការណែនាំ** មិនមែនដំណោះស្រាយ។ ពេលបានកូដ បូររឿងមួយ ហើយទស្សន៍ទាយលទ្ធផលមុនពេលដំណើរការ។

LAB 9 · CONSOLE PRACTICE Lab ៩ · អនុវត្តក្នុង Console

1. Create `js/practice.js`, load it with `defer` in a test page, and print your name and age with a template literal.
   បង្កើត practice.js ហើយបោះពុម្ពឈ្មោះ និងអាយុ។
2. Write `grade(score)` that returns A (85+), B (70+), C (50+) or F, and test it with five scores.
   សរសេរអនុគមន៍ grade(score) ហើយសាកល្បងជាមួយពិន្ទុប្រាំ។
3. Make an array of five project objects with `title`, `year` and `tech`. Use `filter` to list projects from this year and `map` to make an array of titles.
   បង្កើតអារេគម្រោងប្រាំ ហើយប្រើ filter និង map។
4. Use `reduce` to count how many projects use CSS.
   ប្រើ reduce ដើម្បីរាប់គម្រោងដែលប្រើ CSS។
5. Break your code on purpose (misspell a variable). Read the error message and explain it to a classmate.
   ធ្វើឱ្យកូដខូចដោយចេតនា ហើយពន្យល់សារកំហុស។

Exercises

លំហាត់

1. Predict the output: `console.log(typeof null, typeof [], 3 + "3", "10" - 4)`.
   ទស្សន៍ទាយលទ្ធផល។
2. Write `khrToUsd(riel)` using an exchange rate stored in a constant. Why keep the rate in one place?

សរសេរអនុគមន៍បំប្លែងរៀលទៅដុល្លារ។

3. Rewrite the `for` loop in Listing 9.2 with `for...of` over an array of order numbers.
   សរសេរវង្វិលជុំឡើងវិញដោយ for...of។
4. From the menu in Listing 9.3, use one chain of methods to get the names of drinks under $2, in upper case.
   ប្រើ method ជាខ្សែដើម្បីយកឈ្មោះភេសជ្ជៈក្រោម $2។
5. Explain why `const cart = []; cart.push("tea");` works even though `cart` is a constant.
   ពន្យល់ហេតុអ្វី push ដំណើរការលើ const។

SESSION 10

THE DOM AND EVENTS

DOM និងព្រឹត្តិការណ៍

IN THIS SESSION ក្នុងវគ្គនេះ

- Describe the DOM as a tree of objects that JavaScript can change
- Select elements and change their text, classes and attributes
- Create and remove elements safely
- Respond to clicks, input and form submission
- ពិពណ៌នា DOM ជាដើមឈើនៃវត្ថុដែល JavaScript អាចផ្លាស់ប្តូរបាន
- ជ្រើសធាតុ និងប្តូរអត្ថបទ class និង attribute
- បង្កើត និងលុបធាតុដោយសុវត្ថិភាព
- ឆ្លើយតបនឹងការចុច ការវាយ និងការដាក់ស្នើទម្រង់

10.1 What is the DOM?

តើ DOM ជាអ្វី?

When the browser reads your HTML, it builds the **Document Object Model** (DOM): a tree of objects in memory, one for each element, with `document` at the root. The *Elements* panel in DevTools shows this live tree, not your original file. JavaScript reads and changes the tree, and the browser redraws the page to match. That is how every interactive website works.

To change something, first select it. `document.querySelector(".card")` returns the first element that matches a CSS selector, and `document.querySelectorAll(".card")` returns all of them as a list you can loop over. Then change it: `textContent` sets the text, `classList.add`, `remove` and `toggle` change classes, and `setAttribute` sets attributes.

HTML INDEX.HTML LISTING 10.1 · EX/S10_DOM

```html
<h1 id="greeting">Hello</h1>
<ul id="list"></ul>

<script>
  const title = document.querySelector("#greeting");
  title.textContent = "Welcome to Mekong Coffee";
  title.style.color = "#1d4ed8";

  const drinks = ["Iced coffee", "Green tea", "Coconut latte"];
  const list = document.querySelector("#list");
  for (const d of drinks) {
    const li = document.createElement("li");
    li.textContent = d;
    list.append(li);
  }
</script>
```

RESULT IN THE BROWSER · លទ្ធផល CHROMIUM



560px wide

ពេល browser អាន HTML វាសាងសង់ **Document Object Model (DOM)**៖ ដើមឈើនៃវត្ថុក្នុងអង្គចងចាំ មួយសម្រាប់ធាតុនីមួយៗ ដោយមាន `document` ជាឫស។ ផ្ទាំង Elements ក្នុង DevTools បង្ហាញដើមឈើផ្ទាល់នេះ មិនមែនឯកសារដើមរបស់អ្នកទេ។ JavaScript អាន និងផ្លាស់ប្តូរដើមឈើ ហើយ browser គូរទំព័រឡើងវិញ។ ដើម្បីផ្លាស់ប្តូរអ្វីមួយ ត្រូវជ្រើសវាជាមុនដោយ `querySelector` ឬ `querySelectorAll` រួចប្តូរដោយ `textContent`, `classList` ឬ `setAttribute` ។

KEY IDEA · SAFETY គំនិតសំខាន់ · សុវត្ថិភាព

Use `textContent` for text that comes from users or from the internet. `innerHTML` treats the text as HTML, so a comment containing a `<script>` or an image with an `onerror` attribute could run an attacker's code in your page. This attack is called **cross-site scripting** (XSS). Build elements with `createElement` and `textContent` instead.

ប្រើ `textContent` សម្រាប់អត្ថបទដែលមកពីអ្នកប្រើ ឬអ៊ីនធឺណិត។ `innerHTML` ចាត់ទុកអត្ថបទជា HTML ដូច្នេះមតិយោបល់ដែលមានកូដអាចដំណើរការកូដរបស់អ្នកវាយប្រហារ។ ការវាយប្រហារនេះហៅថា **cross-site scripting (XSS)**។

10.2 Events

ព្រឹត្តិការណ៍

An **event** is something that happens: a click, a key press, a form submission, the page finishing loading. `element.addEventListener("click", handler)` tells the browser to call the **handler** function every time that event happens on that element. Common events are `click`, `input` (the value of a field changed), `submit`, `keydown` and `DOMContentLoaded`.

A clean pattern for small apps is: keep your data in variables (the **state**), change the state in event handlers, then call one `render` function that updates the page from the state. Listing 10.2 follows this pattern. The screenshot was taken after the + button was clicked three times and the − button once.

HTML INDEX.HTML LISTING 10.2 · EX/S10_COUNTER

```html
<p>Cups: <output id="count">0</output></p>
<button id="minus" type="button">-</button>
<button id="add" type="button">+</button>

<script>
  let count = 0;
  const out = document.querySelector("#count");

  function render() { out.textContent = count; }

  document.querySelector("#add").addEventListener("click", () => {
    count++;
    render();
  });
  document.querySelector("#minus").addEventListener("click", () => {
    if (count > 0) count--;
    render();
  });
</script>
```

RESULT IN THE BROWSER · លទ្ធផល CHROMIUM

Cups: 2

- +

560px wide

ព្រឹត្តិការណ៍ (event) គឺជាអ្វីដែលកើតឡើង៖ ការចុច ការចុចគ្រាប់ចុច ការដាក់ស្នើទម្រង់។ `addEventListener("click", handler)` ប្រាប់ browser ឱ្យហៅអនុគមន៍ handler រាល់ពេលព្រឹត្តិការណ៍នោះកើតឡើង។ គំរូស្អាតសម្រាប់កម្មវិធីតូចៗ៖ រក្សាទិន្នន័យក្នុងអថេរ (state) ប្តូរ state ក្នុង handler រួចហៅអនុគមន៍ render មួយដើម្បីធ្វើបច្ចុប្បន្នភាពទំព័រ។ របថតអេក្រង់ត្រូវបានថតក្រោយពេលចុច + បីដង និង – មួយដង។

Notice the buttons are real `<button>` elements. A clickable `<div>` looks the same with CSS, but it cannot be reached with the Tab key and is not announced as a button by screen readers. Native elements give you accessibility for free.

សូមកត់សម្គាល់ថាប៊ូតុងជាធាតុ `<button>` ពិត។ `<div>` ដែលចុចបានមើលទៅដូចគ្នា ប៉ុន្តែមិនអាចទៅដល់ដោយគ្រាប់ចុច Tab ហើយកម្មវិធីអានអេក្រង់មិនប្រកាសថាជាប៊ូតុងទេ។

10.3 Forms and creating elements

ទម្រង់ និងការបង្កើតធាតុ

Listen for `submit` on the form, not `click` on the button: `submit` also fires when the user presses Enter, and the browser's built-in validation runs first. The handler receives an **event object**; `event.preventDefault()` stops the normal behaviour, which is sending the form and reloading the page. Then read the value, create a new element, and add it to the page with `append`. Listing 10.3 is a complete to-do list in about twenty lines. The screenshot shows three tasks added, with the first one clicked to mark it done.

HTML INDEX.HTML LISTING 10.3 · EX/S10_TODO

```
<form id="todo-form">
  <label for="task">New task</label>
  <input id="task" required>
  <button>Add</button>
</form>
<ul id="tasks"></ul>

<style> .done { text-decoration: line-through; color: #888; } </style>
<script>
  const form = document.querySelector("#todo-form");
  const input = document.querySelector("#task");
  const tasks = document.querySelector("#tasks");

  form.addEventListener("submit", (event) => {
    event.preventDefault();             // stop the page reload
    const li = document.createElement("li");
    li.textContent = input.value.trim();
    li.addEventListener("click", () => li.classList.toggle("done"));
    tasks.append(li);
    input.value = "";
    input.focus();
  });
</script>
```

RESULT IN THE BROWSER · លទ្ធផល CHROMIUM

New task Add

- ~~Finish HTML lab~~
- Read chapter 10
- Push to GitHub

560px wide

ស្តាប់ព្រឹត្តិការណ៍ `submit` លើទម្រង់ មិនមែន click លើប៊ូតុងទេ ព្រោះ submit ក៏កើតឡើងពេលអ្នកប្រើចុច Enter ហើយការផ្ទៀងផ្ទាត់របស់ browser ដំណើរការមុន។ `event.preventDefault()` បញ្ឈប់

ឥរិយាបថធម្មតា គឺការផ្ទើទម្រង់ និងផ្ទុកទំព័រឡើងវិញ។ រួចអានតម្លៃ បង្កើតធាតុថ្មី ហើយបន្ថែមវាទៅទំព័រដោយ `append` ។ Listing 10.3 ជាបញ្ជីការងាររពេញលេញក្នុងប្រហែល ២០ បន្ទាត់។

IN THE AI ERA ក្នុងយុគសម័យ AI

AI assistants are very good at writing small interactive features, and very good at hiding problems inside them. When you receive DOM code, review it like a teacher: Does it use `textContent` rather than `innerHTML` for user text? Are clickable things real buttons or links? Does the form still work with the keyboard? Does it handle empty input? Ask the assistant to explain its own code line by line; mistakes often appear in the explanation.

AI ពូកែសរសេរមុខងារអន្តរកម្មតូចៗ ហើយក៏ពូកែលាក់បញ្ហាក្នុងនោះដែរ។ ពិនិត្យកូដ DOM ដូចគ្រូ៖ ប្រើ textContent ជំនួស innerHTML សម្រាប់អត្ថបទអ្នកប្រើ? អ្វីដែលចុចបានជាប៊ូតុង ឬតំណពិត? ទម្រង់ដំណើរការជាមួយក្តារចុច? គ្រប់គ្រងការបញ្ចូលទទេ? សុំឱ្យ AI ពន្យល់កូដរបស់វាម្តងមួយបន្ទាត់។

LAB 10 · INTERACTIVE PORTFOLIO FEATURES Lab ១០ · មុខងារអន្តរកម្មសម្រាប់ផលប័ត្រ

1. Add a menu button that shows and hides the navigation on small screens by toggling a class. Update `aria-expanded` on the button.
   បន្ថែមប៊ូតុងម៉ឺនុយដែលបង្ហាញ/លាក់នាវីហ្គេសិនលើអេក្រង់តូច។
2. On the contact page, show a live character counter under the message box using the `input` event.
   បង្ហាញការរាប់តួអក្សរផ្ទាល់ក្រោមប្រអប់សារ។
3. On submit, prevent the reload and show a thank-you message in a `<p role="status">` so screen readers announce it.
   ពេល submit បង្ហាញសារអរគុណ។
4. Add filter buttons on the projects page (All, HTML, CSS, JS) that hide cards that do not match.
   បន្ថែមប៊ូតុងចម្រោះនៅទំព័រគម្រោង។
5. Use every feature with the keyboard only.
   ប្រើមុខងារទាំងអស់ដោយក្តារចុចតែប៉ុណ្ណោះ។

Exercises

លំហាត់

1. What is the difference between `querySelector` and `querySelectorAll`?
   តើ querySelector និង querySelectorAll ខុសគ្នាដូចម្តេច?

2. Why is `li.innerHTML = userInput` dangerous? Rewrite it safely.
   ហេតុអ្វីកូដនេះគ្រោះថ្នាក់? សរសេរវាឡើងវិញដោយសុវត្ថិភាព។
3. Add a “Clear done” button to Listing 10.3 that removes every task with the class `done`.
   បន្ថែមប៊ូតុង «Clear done» ទៅ Listing 10.3។
4. What would happen in Listing 10.3 without `event.preventDefault()`?
   តើនឹងមានអ្វីកើតឡើងបើគ្មាន preventDefault()?
5. Change Listing 10.2 so the count can never go above 10, and disable the + button at 10.
   កែ Listing 10.2 កុំឱ្យលើស 10 ហើយបិទប៊ូតុង + នៅ 10។

SESSION 11

JSON, FETCH AND STORING DATA

JSON, Fetch និងការរក្សាទុកទិន្នន័យ

IN THIS SESSION ក្នុងវគ្គនេះ

- Read and write JSON, the format most web APIs use
- Load data with `fetch`, `async` and `await`, and handle errors
- Render a list on the page from data
- Save small settings in the browser with `localStorage`
- Keep secrets out of front-end code

- អាន និងសរសេរ JSON ដែលជាទម្រង់ដែល API ភាគច្រើនប្រើ
- ទាញយកទិន្នន័យដោយ fetch, async និង await ហើយគ្រប់គ្រងកំហុស
- បង្ហាញបញ្ជីលើទំព័រពីទិន្នន័យ
- រក្សាទុកការកំណត់តូចៗក្នុង browser ដោយ localStorage
- មិនដាក់ព័ត៌មានសម្ងាត់ក្នុងកូដ front-end

11.1 JSON

JSON

JSON (JavaScript Object Notation) is a text format for data that looks like JavaScript objects and arrays [19]. Most web APIs send and receive JSON. It is stricter than JavaScript: keys must be in double quotes, strings use double quotes, and there are no comments or trailing commas. `JSON.parse(text)` turns JSON text into a JavaScript value, and `JSON.stringify(value)` turns a value into JSON text. Invalid JSON throws an error, which you can catch with `try…catch`.

JS SCRIPT.JS LISTING 11.1 · EX/S11_JSON

```
const text = '{"shop":"Mekong Coffee","open":true,"drinks":["latte","tea"]}';

const data = JSON.parse(text);          // text  -> object
console.log(data.shop, data.drinks.length);

data.drinks.push("iced coffee");
const back = JSON.stringify(data);      // object -> text
console.log(back);

try {
  JSON.parse("{ bad json }");
} catch (error) {
  console.log("Error:", error.name);
}
```

CONSOLE OUTPUT · លទ្ធផលក្នុង CONSOLE

```
Mekong Coffee 2
{"shop":"Mekong Coffee","open":true,"drinks":["latte","tea","iced coffee"]}
Error: SyntaxError
```

JSON ជាទម្រង់អត្ថបទសម្រាប់ទិន្នន័យ ដែលមើលទៅដូចវត្ថុ និងអារេ JavaScript។ API របភាគច្រើនផ្ញើ និងទទួល JSON។ វាតឹងរ៉ឹងជាង JavaScript៖ key ត្រូវនៅក្នុងសញ្ញាសម្រង់ទ្វេ string ប្រើសញ្ញាសម្រង់ទ្វេ ហើយគ្មាន comment ឬក្បៀសនៅចុង។ `JSON.parse` បំប្លែងអត្ថបទ JSON ទៅជាតម្លៃ JavaScript ហើយ `JSON.stringify` បំប្លែងតម្លៃទៅជាអត្ថបទ JSON។ JSON មិនត្រឹមត្រូវបង្កកំហុស ដែលអ្នកអាចចាប់បានដោយ `try…catch` ។

11.2 Fetching data

ការទាញយកទិន្នន័យ

`fetch(url)` sends an HTTP request from JavaScript [20]. A network request takes time, so `fetch` does not return the data immediately; it returns a **promise**, a placeholder for a value that will arrive later. Inside an `async` function you can write `await` before a promise to pause that function until the value arrives, while the rest of the page keeps working.

Two checks matter. First, `fetch` only fails by itself when the network fails; a 404 or 500 response still “succeeds”, so check `response.ok`. Second, always show the user what is happening: a loading message, the result, or a friendly error. Listing 11.2 loads a local `menu.json` file.

HTML INDEX.HTML LISTING 11.2 · EX/S11_FETCH

```html
<h2>Menu</h2>
<p id="status">Loading...</p>
<ul id="menu"></ul>

<script>
  async function loadMenu() {
    const status = document.querySelector("#status");
    try {
      const response = await fetch("menu.json");
      if (!response.ok) throw new Error(`HTTP ${response.status}`);
      const items = await response.json();

      const list = document.querySelector("#menu");
      for (const item of items) {
        const li = document.createElement("li");
        li.textContent = `${item.name} - $${item.price.toFixed(2)}`;
        list.append(li);
      }
      status.textContent = `${items.length} drinks loaded`;
    } catch (error) {
      status.textContent = "Sorry, the menu could not load.";
      console.error(error);
    }
  }
  loadMenu();
</script>
```

RESULT IN THE BROWSER · លទ្ធផល CHROMIUM

Menu

4 drinks loaded

- Iced coffee - $1.50
- Black coffee - $1.25
- Coconut latte - $2.25
- Lemongrass tea - $1.25

560px wide

`fetch(url)` ផ្ញើសំណើ HTTP ពី JavaScript។ សំណើតាមបណ្តាញត្រូវការពេល ដូច្នេះ fetch ត្រឡប់ **promise** ដែលជាកន្លែងរង់ចាំតម្លៃដែលនឹងមកដល់ពេលក្រោយ។ ក្នុងអនុគមន៍ `async` អ្នកអាចសរសេរ `await` ដើម្បីរង់ចាំតម្លៃ ខណៈដែលចំព័រនៅតែដំណើរការ។ ពិនិត្យពីរចំណុច៖ fetch បរាជ័យដោយខ្លួនឯងតែពេលបណ្តាញខូច ចំណែកចម្លើយ 404 ឬ 500 នៅតែ «ជោគជ័យ» ដូច្នេះត្រូវពិនិត្យ `response.ok` ។ ហើយត្រូវបង្ហាញអ្នកប្រើពីអ្វីដែលកំពុងកើតឡើង៖ សារកំពុងផ្ទុក លទ្ធផល ឬសារកំហុសដែលស្រួលយល់។

PRACTICAL NOTE កំណត់សម្គាល់អនុវត្ត

Browsers block `fetch` from pages opened as files (`file://`). Run a small local web server instead, for example the Live Server extension in VS Code, or `python -m http.server` in your project folder, then open `http://localhost:8000`. Requests to other websites also need that website's permission, a rule called **CORS**; public APIs designed for browsers allow it.

Browser ហាមឃាត់ fetch ពីទំព័រដែលបើកជាឯកសារ (file://)។ សូមដំណើរការ web server តូចមួយ ដូចជា Live Server ក្នុង VS Code ឬ `python -m http.server` រួចបើក http://localhost:8000។ សំណើទៅគេហទំព័រផ្សេងក៏ត្រូវការការអនុញ្ញាតពីគេហទំព័រនោះ ដែលជាច្បាប់ហៅថា **CORS**។

11.3 Saving data in the browser

ការរក្សាទុកទិន្នន័យក្នុង browser

`localStorage` keeps small pieces of text in the user's browser, even after the tab is closed. It is useful for preferences such as a chosen theme, or a draft that should survive a reload. It stores only strings, so use `JSON.stringify` and `JSON.parse` for objects and arrays. `getItem` returns `null` when a key does not exist. The data stays on that one device and browser, any script on your page can read it, and the user can clear it at any time, so never store passwords or anything important only there.

JS SCRIPT.JS LISTING 11.3 · EX/S11_STORAGE

```js
// Save a small setting in the browser
localStorage.setItem("theme", "dark");
console.log(localStorage.getItem("theme"));

// Objects must be turned into text first
const cart = [{ name: "Latte", qty: 2 }];
localStorage.setItem("cart", JSON.stringify(cart));

const saved = JSON.parse(localStorage.getItem("cart"));
console.log(saved[0].name, saved[0].qty);

localStorage.removeItem("theme");
console.log(localStorage.getItem("theme"));   // null when missing
```

CONSOLE OUTPUT · លទ្ធផលក្នុង CONSOLE

```
dark
Latte 2
null
```

`localStorage` រក្សាទុកអត្ថបទតូចៗក្នុង browser របស់អ្នកប្រើ សូម្បីតែក្រោយបិទផ្ទាំង។ វាមានប្រយោជន៍សម្រាប់ការកំណត់ដូចជាទម្រង់ពណ៌ ឬសេចក្តីព្រាងដែលគួររក្សាក្រោយ reload។ វារក្សាតែ string ដូច្នេះប្រើ

JSON.stringify និង JSON.parse សម្រាប់វត្ថុ។ `getItem` ត្រឡប់ null ពេលគ្មាន key។ ទិន្នន័យនៅតែលើឧបករណ៍ និង browser នោះ ស្ត្រីបណាក៏អានបាន ហើយអ្នកប្រើអាចលុបវាបានគ្រប់ពេល ដូច្នេះកុំរក្សាទុកពាក្យសម្ងាត់ ឬអ្វីដែលសំខាន់នៅទីនោះតែមួយ។

11.4 APIs and secrets

API និងព័ត៌មានសម្ងាត់

An **API** (Application Programming Interface) is a door that a program offers to other programs. Weather services, maps, payment providers and AI models all offer web APIs, and many require an **API key** to identify who is calling. Everything in front-end code can be read by anyone who opens DevTools, so a secret key placed in your JavaScript is no longer secret: someone can copy it and use your account or your money. Keys belong on a server, which calls the API for you. For this course, use public APIs that need no key, or your own JSON files.

API គឺជាទ្វារដែលកម្មវិធីមួយផ្តល់ឱ្យកម្មវិធីផ្សេងទៀត។ សេវាអាកាសធាតុ ផែនទី ការទូទាត់ និងម៉ូដែល AI សុទ្ធតែមាន API ហើយច្រើនតម្រូវឱ្យមាន **API key**។ អ្វីៗក្នុងកូដ front-end អាចត្រូវបានអានដោយនរណាម្នាក់ដែលបើក DevTools ដូច្នេះ key សម្ងាត់ក្នុង JavaScript របស់អ្នកលែងសម្ងាត់ទៀតហើយ៖ គេអាចចម្លងវា ហើយប្រើគណនី ឬលុយរបស់អ្នក។ Key ត្រូវនៅលើ server។ សម្រាប់មុខវិជ្ជានេះ សូមប្រើ API សាធារណៈ ដែលមិនត្រូវការ key ឬឯកសារ JSON ផ្ទាល់ខ្លួន។

IN THE AI ERA ក្នុងយុគសម័យ AI

Many students now want to put an AI chatbot on their site. The safe pattern is the one above: the browser calls *your* server, and your server calls the AI provider with a key stored in an environment variable. AI-generated tutorials sometimes put the key directly in front-end code “for simplicity”. Treat that as a bug. Also check any API that an assistant suggests: it may have changed, require payment, or not exist.

និស្សិតជាច្រើនចង់ដាក់ chatbot AI លើគេហទំព័រ។ គំរូសុវត្ថិភាពគឺ៖ browser ហៅ server របស់អ្នក ហើយ server ហៅអ្នកផ្តល់ AI ដោយ key ដែលរក្សាទុកក្នុង environment variable។ ការបង្រៀនដែល AI បង្កើតពេលខ្លះដាក់ key ក្នុងកូដ front-end «ដើម្បីងាយស្រួល»។ ចាត់ទុកវាជាកំហុស។ ពិនិត្យ API ដែល AI ស្នើផងដែរ ព្រោះវាអាចបានផ្លាស់ប្តូរ ត្រូវបង់ប្រាក់ ឬមិនមានពិតប្រាកដ។

LAB 11 · PROJECTS FROM JSON Lab ១១ · គម្រោងពី JSON

1. Move your project data into `data/projects.json`: title, year, tech (an array), description and link.
   ផ្លាស់ទិន្នន័យគម្រោងទៅ data/projects.json។

2. On `projects.html`, fetch the file and build the cards with `createElement`. Show "Loading…" first and a friendly error if it fails.
   ទាញយកឯកសារ ហើយបង្កើតកាតដោយ createElement។
3. Rename the file on purpose to test your error message, then fix it.
   ប្តូរឈ្មោះឯកសារដោយចេតនាដើម្បីសាកល្បងសារកំហុស។
4. Add a theme switch (light, dark, system) and remember the choice in `localStorage`.
   បន្ថែមកុងតាក់ប្តូរទម្រង់ពណ៌ ហើយចងចាំជម្រើសក្នុង localStorage។
5. Optional: show data from a free public API that needs no key, with the source credited on the page.
   ជាជម្រើស៖ បង្ហាញទិន្នន័យពី API សាធារណៈឥតគិតថ្លៃ។

Exercises

លំហាត់

1. Find the three errors in this JSON: `{name: 'Latte', "price": 2.25,}`.
   រកកំហុសបីក្នុង JSON នេះ។
2. Why does a 404 response not go to the `catch` block in Listing 11.2 by itself?
   ហេតុអ្វីចម្លើយ 404 មិនទៅ catch ដោយខ្លួនឯង?
3. Write a function that saves and loads a shopping cart (an array of objects) with `localStorage`.
   សរសេរអនុគមន៍រក្សាទុក និងផ្ទុករទេះទិញទំនិញ។
4. A tutorial puts `const API_KEY = "sk-…"` in `script.js`. Explain the risk and the fix.
   ពន្យល់ហានិភ័យ និងដំណោះស្រាយ។
5. What will the page in Listing 11.2 show while the request is still loading? Why does that matter on a slow network?
   តើទំព័របង្ហាញអ្វីពេលកំពុងផ្ទុក? ហេតុអ្វីវាសំខាន់?

SESSION 12

QUALITY, PUBLISHING AND THE AI-ASSISTED WORKFLOW

គុណភាព ការផ្សព្វផ្សាយ និងលំហូរការងារជាមួយ AI

IN THIS SESSION ក្នុងវគ្គនេះ

- Run an accessibility, performance and SEO check before launch
- Use Lighthouse and the HTML validator to find problems
- Put a project under version control with Git
- Publish a site free with GitHub Pages
- Use a responsible five-step workflow with AI assistants

- ពិនិត្យភាពងាយស្រួលប្រើ ល្បឿន និង SEO មុនពេលដាក់ឱ្យដំណើរការ
- ប្រើ Lighthouse និង validator ដើម្បីរកបញ្ហា
- ដាក់គម្រោងក្រោមការគ្រប់គ្រងកំណែដោយ Git
- ផ្សព្វផ្សាយគេហទំព័រដោយឥតគិតថ្លៃជាមួយ GitHub Pages
- ប្រើលំហូរការងារប្រាំជំហានជាមួយជំនួយការ AI ប្រកបដោយការទទួលខុសត្រូវ

12.1 An accessibility check

ការពិនិត្យភាពងាយស្រួលប្រើ

Automatic tools find only part of the problems, so combine them with a short manual check. Listing 12.1 collects several small techniques from this book: a **skip link** that lets keyboard users jump past the header (it appears only when focused), meaningful and empty `alt` text, an icon-only button with an `aria-label` so screen readers announce “Add to cart”, a 44 px touch target, and a labelled input.

HTML INDEX.HTML LISTING 12.1 · EX/S12_A11Y

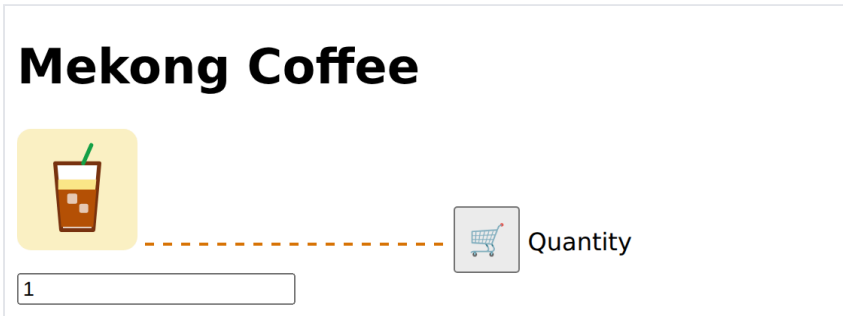
```html
<style>
  body{font-family:system-ui,sans-serif}
  .icon-btn{font-size:20px;min-width:44px;min-height:44px}
  .skip{position:absolute;left:-999px}
  .skip:focus{left:8px}
</style>

<a class="skip" href="#main">Skip to content</a>
<header><h1>Mekong Coffee</h1></header>
<main id="main">
  <img src="cup.svg" alt="Iced coffee in a glass with ice" width="80" height="80">
  <img src="divider.svg" alt="" width="200" height="8">  <!-- decorative -->

  <button class="icon-btn" type="button" aria-label="Add to cart">
    &#128722;
  </button>

  <label for="qty">Quantity</label>
  <input id="qty" type="number" min="1" value="1">
</main>
```

RESULT IN THE BROWSER · លទ្ធផល CHROMIUM



560px wide

Use ARIA attributes such as `aria-label` only when native HTML cannot express what you need. The first rule of ARIA is to prefer a real `<button>`, `<label>` or `<nav>`, because they already work with keyboards and assistive technology.

- ☐ Can I reach and use everything with Tab, Shift+Tab, Enter and Space? Is focus always visible?
- ☐ Does every page have a title, one `<h1>` and a logical heading order?
- ☐ Do meaningful images have `alt` text and decorative ones `alt=""`?
- ☐ Does all text reach a 4.5:1 contrast ratio, in light and dark themes?
- ☐ Does the page still work when zoomed to 200% and at 320 px wide?
- ☐ Do all form inputs have labels, and are errors explained in text?

ឧបករណ៍ស្វ័យប្រវត្តិរកឃើញតែផ្នែកខ្លះនៃបញ្ហា ដូច្នេះត្រូវផ្សំវាជាមួយការពិនិត្យដោយដៃខ្លីមួយ។ Listing 12.1 ប្រមូលបច្ចេកទេសតូចៗ៖ **skip link** ដែលអនុញ្ញាតឱ្យអ្នកប្រើក្តារចុចរំលងក្បាលទំព័រ alt ដែលមានន័យ និង alt ទទេ ប៊ូតុងរូបតំណាងដែលមាន `aria-label` គោលដៅចុច 44px និង input ដែលមាន label។ ប្រើ ARIA តែពេល HTML ធម្មតាមិនអាចបង្ហាញអ្វីដែលអ្នកត្រូវការ។ បញ្ជីត្រួតពិនិត្យ៖ ប្រើបានដោយក្តារចុច ការផ្តោតមើលឃើញ ចំណងជើងសមហេតុផល alt ត្រឹមត្រូវ កម្រិតពណ៌ផ្ទុយ 4.5:1 ដំណើរការនៅ zoom 200% និងទទឹង 320px ហើយ input ទាំងអស់មាន label។

12.2 Lighthouse, performance and SEO

Lighthouse ល្បឿន និង SEO

Lighthouse is a free tool built into Chrome DevTools. It audits a page in four categories: Performance, Accessibility, Best Practices and SEO, and explains each failing audit [21]. Run it in an incognito window so browser extensions do not affect the result, and test the mobile setting. Treat the score as a guide, not a goal: a page can score 100 and still be confusing to use.

The biggest performance wins for small sites are simple: resize and compress images (Session 7), give images width and height, load scripts with `defer`, and avoid large libraries when a few lines of your own JavaScript will do. For **search engine optimisation** (SEO), the basics are the same as good HTML: a unique `<title>` and a `<meta name="description">` on every page, clear headings, descriptive link text and real text content.

Lighthouse ជាឧបករណ៍ឥតគិតថ្លៃក្នុង Chrome DevTools។ វាពិនិត្យទំព័រក្នុងបួនប្រភេទ៖ Performance, Accessibility, Best Practices និង SEO ហើយពន្យល់ការពិនិត្យនីមួយៗដែលបរាជ័យ។ ដំណើរការវាក្នុងបង្អួច incognito ហើយសាកល្បងការកំណត់ទូរស័ព្ទ។ ចាត់ទុកពិន្ទុជាការណែនាំ មិនមែនជាគោលដៅ។ ការកែលម្អល្បឿនធំៗសម្រាប់គេហទំព័រតូចគឺសាមញ្ញ៖ បង្រួម និងបង្គាប់រូបភាព ដាក់ width និង height ផ្ទុកស្ក្រីបដោយ defer និងជៀសវាងបណ្ណាល័យធំៗ។ សម្រាប់ SEO មូលដ្ឋានគឺដូចគ្នានឹង HTML ល្អ៖ title និង meta description ដាច់ដោយឡែកលើទំព័រនីមួយៗ ចំណងជើងច្បាស់ និងអត្ថបទតំណដែលពិពណ៌នា។

12.3 Version control with Git

ការគ្រប់គ្រងកំណែដោយ Git

Git records snapshots of your project, called **commits**, so you can see what changed, undo mistakes and work with others. **GitHub** is a website that hosts Git repositories online. The basic cycle in a terminal is short:

```
git init                    # start tracking this folder (once)
git add .                   # choose all changed files
```

```
git commit -m "Add contact form"
git push                      # send commits to GitHub
```

Write commit messages that say what the change does, in the present tense. Commit small, working steps often: one feature or one fix per commit. Never commit passwords or API keys; once pushed to a public repository, consider them leaked and replace them.

> **Git** កត់ត្រារូបថតនៃគម្រោងរបស់អ្នក ហៅថា **commit** ដូច្នេះអ្នកអាចមើលអ្វីដែលបានប្តូរ មិនធ្វើវិញនូវកំហុស និងធ្វើការជាមួយអ្នកដទៃ។ **GitHub** ជាគេហទំព័រដែលរក្សាទុក repository Git លើអ៊ីនធឺណិត។ សរសេរសារ commit ដែលប្រាប់ថាការផ្លាស់ប្តូរធ្វើអ្វី ហើយ commit ជាជំហានតូចៗដែលដំណើរការញឹកញាប់។ កុំ commit ពាក្យសម្ងាត់ ឬ API key ឡើយ។

12.4 Publishing with GitHub Pages

ការផ្សព្វផ្សាយជាមួយ GitHub Pages

GitHub Pages hosts static sites (HTML, CSS, JavaScript and images) for free from a GitHub repository [22]. For a personal site, create a repository named `username.github.io`, replacing *username* with your GitHub username. Push your files with `index.html` at the top level. In the repository, open **Settings** → **Pages**, choose **Deploy from a branch**, and select your main branch. After a few minutes your site is live at `https://username.github.io`; GitHub notes that changes can take up to ten minutes to appear.

After publishing, test the live site on a phone and run Lighthouse again. Check that every link works, because file names are case-sensitive on the server: `About.html` and `about.html` are different files.

> **GitHub Pages** ផ្ទុកគេហទំព័រ static (HTML, CSS, JavaScript និងរូបភាព) ដោយឥតគិតថ្លៃពី repository GitHub។ សម្រាប់គេហទំព័រផ្ទាល់ខ្លួន បង្កើត repository ឈ្មោះ `username.github.io` (ប្តូរ username ជាឈ្មោះអ្នកប្រើ GitHub របស់អ្នក)។ Push ឯកសារដែលមាន index.html នៅថតខាងលើបង្អស់។ បើក **Settings** → **Pages** ជ្រើស **Deploy from a branch** ហើយជ្រើស branch ចម្បង។ ប៉ុន្មាននាទីក្រោយ គេហទំព័ររបស់អ្នកដំណើរការនៅ `https://username.github.io` ។ GitHub កត់សម្គាល់ថាការផ្លាស់ប្តូរអាចត្រូវការរហូតដល់ ១០ នាទី។ បន្ទាប់មកសាកល្បងលើទូរស័ព្ទ និងពិនិត្យតំណទាំងអស់ ព្រោះឈ្មោះឯកសារលើ server ប្រកាន់អក្សរធំតូច។

12.5 A responsible AI-assisted workflow

លំហូរការងារជាមួយ AI ប្រកបដោយការទទួលខុសត្រូវ

Professional developers increasingly use AI assistants every day. The difference between a beginner and a professional is not whether they use AI, but whether they stay in control of the result. This five-step loop works for any feature in this book:

PLAN	DRAFT	REVIEW	TEST	RECORD
Write what you need, for whom, and the constraints. សរសេរតម្រូវការ និងលក្ខខណ្ឌ	Ask the AI for a small piece, with context. សុំផ្នែកតូចមួយពី AI	Read every line. Ask it to explain anything unclear. អានគ្រប់បន្ទាត់	Browser, keyboard, phone, validator, Lighthouse. សាកល្បងគ្រប់ផ្នែក	Commit, and note how AI was used. Commit និងកត់ត្រាការប្រើ AI

អ្នកអភិវឌ្ឍអាជីពកាន់តែច្រើនប្រើជំនួយការ AI ជារៀងរាល់ថ្ងៃ។ ភាពខុសគ្នារវាងអ្នកចាប់ផ្តើម និងអ្នកអាជីពមិនមែនថាពួកគេប្រើ AI ឬអត់នោះទេ ប៉ុន្តែថាពួកគេនៅតែគ្រប់គ្រងលទ្ធផលឬអត់។ រង្វិលប្រាំជំហាននេះដំណើរការសម្រាប់មុខងារណាមួយ៖ រៀបចំផែនការ ព្រាង ពិនិត្យ សាកល្បង និងកត់ត្រា។

IN THE AI ERA · WHAT AI DOES NOT REPLACE អ្វីដែល AI មិនជំនួស

- **Understanding users**: talking to real people and watching them use your site.
- **Judgement**: choosing what to build, and what to leave out.
- **Responsibility**: for accessibility, privacy, security, licences and honesty.
- **Fundamentals**: the HTML, CSS and JavaScript in this book, which let you check any draft, from a person or a machine.

ការយល់ពីអ្នកប្រើ ការវិនិច្ឆ័យ ការទទួលខុសត្រូវ (ភាពងាយស្រួលប្រើ ឯកជនភាព សុវត្ថិភាព អាជ្ញាប័ណ្ណ ភាពស្មោះត្រង់) និងមូលដ្ឋានគ្រឹះ HTML, CSS, JavaScript ដែលអនុញ្ញាតឱ្យអ្នកពិនិត្យសេចក្តីព្រាងណាមួយ។

LAB 12 · LAUNCH YOUR PORTFOLIO (FINAL PROJECT) Lab ១២ · ដាក់ផលប័ត្រឱ្យដំណើរការ (គម្រោងចុងក្រោយ)

1. Run the accessibility checklist in 12.1 on every page and fix what you find.
   អនុវត្តបញ្ជីត្រួតពិនិត្យលើទំព័រនីមួយៗ ហើយកែបញ្ហា។
2. Add a unique title and meta description to each page. Run Lighthouse (mobile) and fix the top three issues.
   បន្ថែម title និង meta description ហើយកែបញ្ហាធំៗបីពី Lighthouse។
3. Create a GitHub repository, commit your work in small steps and publish with GitHub Pages.
   បង្កើត repository, commit និងផ្សព្វផ្សាយជាមួយ GitHub Pages។
4. Write a short `README.md`: what the site is, the technologies, and an honest note on how you used AI.
   សរសេរ README ខ្លីមួយ រួមទាំងកំណាត់សម្គាល់ស្មោះត្រង់អំពីការប្រើ AI។
5. Present your site in five minutes: one design decision, one bug you fixed, and one thing you would improve. The rubric is in Appendix A.

> បង្ហាញគេហទំព័ររបស់អ្នកក្នុងរយៈពេលប្រាំនាទី។

Exercises

លំហាត់

1. Why should you not rely only on an automatic accessibility score?
   ហេតុអ្វីមិនគួរពឹងតែលើពិន្ទុស្វ័យប្រវត្តិ?
2. When is `aria-label` needed, and when is it better to change the HTML instead?
   ពេលណាត្រូវការ aria-label?
3. Write three good commit messages for changes you made this semester.
   សរសេរសារ commit ល្អបី។
4. Your live site shows no styles, but it works on your laptop. Name two likely causes.
   គេហទំព័រផ្ទាល់គ្មានរចនាប័ទ្ម ប៉ុន្តែដំណើរការលើកុំព្យូទ័រ។ ប្រាប់មូលហេតុពីរ។
5. Write a short AI-use statement for your final project that is honest and specific.
   សរសេរសេចក្តីថ្លែងការណ៍ខ្លីអំពីការប្រើ AI។

APPENDIX A · ANSWERS, HINTS AND PROJECT RUBRIC

ឧបសម្ព័ន្ធ A · ចម្លើយ ការណែនាំ និងតារាងវាយតម្លៃ

Short answers or hints for every exercise. Try each exercise before reading the answer. Open questions have no single correct answer; a sample is given.

> ចម្លើយខ្លី ឬការណែនាំសម្រាប់លំហាត់ទាំងអស់។ សូមសាកល្បងធ្វើលំហាត់ជាមុនសិន មុននឹងអានចម្លើយ។

Session 1

1. The client (your browser) asks for files; the server stores the site and sends the files back.
2. Scheme `https://`; host `shop.example.com`; path `/products/tea.html`; query `?color=green`; fragment `#reviews`.
3. “Not Found”: the requested file does not exist. The server sends it in its HTTP response.
4. Sample: HTML marks up the menu list; CSS sets the brand colours and layout; JavaScript shows “Open now” or “Closed” from the current time.
5. Learning prompt: “Explain what each line does; do not rewrite it.” Non-learning prompt: “Write my page.” The first keeps you doing the thinking.

Session 2

1. Headings describe structure for screen readers, search engines and the page outline. Change size with CSS instead.
2. `<a href="../index.html">Home</a>`
3. (a) The destination, e.g. `alt="Mekong Coffee home"`. (b) `alt=""`. (c) The message of the chart, e.g. “Sales rose steadily from January to June”, with the key numbers.
4. Use `<ul>` for ingredients and `<ol>` for steps, inside headed sections.
5. Tags close in the wrong order (`</strong>` must come before `</p>`), and the image has no `alt` (and no width and height).

Session 3

1. (a) `<article>` (b) `<nav>` (c) `<aside>` (d) `<footer>`.
2. Tables announce rows and columns to screen readers, so layout tables confuse them; they also do not adapt to small screens and are hard to maintain.
3. `<label for="sid">Student ID</label> <input id="sid" name="sid" inputmode="numeric" pattern="[0-9]{8}" required>`

4. `get` for searches and filters that can be bookmarked; `post` for logins, orders and anything private or that changes data.
5. The hint disappears while typing, so users must remember it; placeholders are often low-contrast and are not a reliable label for assistive technology; clicking them does not focus a separate label.

Session 4

1. Open answer. Look for a clear main heading and one obvious primary action, or their absence.
2. Use a darker grey such as `#6B7280` (4.83:1 on white), or a larger, bolder size that only needs 3:1.
3. Error recovery ("help users recognise, diagnose and recover from errors") and user control and freedom.
4. Perceivable: `alt` text. Operable: works with the keyboard. Understandable: clear labels and error messages. Robust: valid HTML that assistive technology can read.
5. Some users cannot tell red from green, and screen readers do not announce colour. Add a text message or an icon.

Session 5

1. `nav ul li a` = 0-0-4; `.nav a` = 0-1-1; `#main .card h2` = 1-1-1. If they target the same element, the last one wins; between the first two, `.nav a` wins.
2. `h1, h2, h3 { font-family: system-ui, sans-serif; }`
3. `rem` follows the user's browser font-size setting, so text scales for people who need larger text.
4. `rgb(29 78 216 / 50%)`
5. Is the stylesheet linked and loaded? Is the rule crossed out by a more specific or later rule? Is there a typo in the selector or property name?

Session 6

1. `content-box`: 300 + 2×10 + 2×2 = 324 px. `border-box`: 300 px.
2. Inline elements ignore width and height. Use `display: inline-block` or `block`.
3. `.wrap { display: flex; justify-content: center; align-items: center; min-height: 300px; }`
4. In a column, `justify-content` works vertically (the main axis) and `align-items` horizontally.
5. With a basis of 0, the free space is shared 2 : 1, so the first item gets two thirds; with `flex: 1 1 0` on both, they are equal.

Session 7

1. `grid-template-columns: 250px 1fr;`
2. `.grid { grid-template-columns: 1fr; }` as the default, then `@media (min-width: 600px) { .grid { grid-template-columns: … } }` with the wider layout.
3. (650 + 10) ÷ (200 + 10) ≈ 3.14, so 3 columns, each (650 − 2×10) ÷ 3 = 210 px wide.
4. Flexbox: a navigation bar, a row of buttons. Grid: a page layout, a photo gallery.
5. On viewports at least 800 px wide the image shows at half the viewport width; otherwise at full width. The browser uses this with `srcset` to choose the file.

Session 8

1. Use `--gap`, or 16 px if `--gap` is not defined (a fallback value).
2. 4vw reaches 32 px (2rem) at an 800 px wide viewport; it stays at 16 px below 400 px.
3. Keyboard users can no longer see where they are on the page.
4. `.card { opacity: 0; transition: opacity 200ms ease; } .card.is-visible { opacity: 1; }`
5. `.card`, `.card__image`, `.card__title`, `.card--sale`.

Session 9

1. `object object 33 6`
2. `const RIEL_PER_USD = 4100; // example only, use the current rate` and `function khrToUsd(riel) { return riel / RIEL_PER_USD; }`. One place means one change when the rate changes.
3. `for (const n of [1, 2, 3]) { console.log(`Order #${n} is ready`); }`
4. `menu.filter(i => i.price < 2).map(i => i.name.toUpperCase())` gives `["ICED COFFEE", "BLACK COFFEE"]`.
5. `const` stops the name from being reassigned; it does not freeze the array's contents.

Session 10

1. `querySelector` returns the first match (or `null`); `querySelectorAll` returns a list of all matches.
2. It allows cross-site scripting. Use `li.textContent = userInput;`
3. `clearBtn.addEventListener("click", () => document.querySelectorAll("#tasks .done").forEach(li => li.remove()));`
4. The form would be submitted and the page reloaded, so the new task would disappear.
5. Use `if (count < 10) count++;` and in `render()` set `addBtn.disabled = count >= 10;`

Session 11

1. The key `name` is not in double quotes; `'Latte'` uses single quotes; there is a trailing comma.
2. `fetch` rejects only on network failure. Any HTTP response, including 404, resolves, so the code checks `response.ok` and throws.
3. `const saveCart = c => localStorage.setItem("cart", JSON.stringify(c));` and `const loadCart = () => JSON.parse(localStorage.getItem("cart")) ?? [];`
4. Anyone can read the key in DevTools and use it. Remove it, revoke it and create a new one, and call the API from a server that keeps the key in an environment variable.
5. “Loading…”. On slow networks it tells users the page is working, so they do not think it is broken.

Session 12

1. Tools cannot judge whether `alt` text is meaningful, whether the reading order makes sense or whether the site is easy to use.
2. For controls without visible text, such as an icon-only button. If you can show a visible text label instead, do that.
3. Sample: “Add dark theme tokens”, “Fix broken link to about page”, “Load projects from JSON”.
4. A path with the wrong letter case (`Style.css` vs `style.css`), or the CSS file was not committed and pushed.
5. Sample: “I used Claude to explain CSS Grid errors and to review my contact form for accessibility. I wrote and tested all final code myself.”

Final project rubric

តារាងរាយតម្លៃគម្រោងចុងក្រោយ

Criterion	What good work shows	Points
Content and structure	Clear purpose, at least four pages, consistent navigation	15
Semantic, valid HTML	Landmarks, heading order, labels; zero validator errors	15
Visual design and CSS	Hierarchy, tokens, components, readable typography	20
Responsive layout	Mobile-first; works from 320 px to desktop	15
Accessibility	Keyboard use, focus, contrast 4.5:1, alt text	15
JavaScript	At least two working interactive features, safe DOM code	10
Process and honesty	Git history, README, live URL, specific AI-use statement	10

APPENDIX B · QUICK REFERENCE

ឧបសម្ព័ន្ធ B · ឯកសារយោងរហ័ស

HTML	Use	CSS	Use
`<h1>…<h6>`	headings	`color`, `background`	colours
`<p>`	paragraph	`font-family`, `font-size`	type
`<a href>`	link	`line-height`	line spacing
`<img src alt>`	image	`margin`, `padding`	spacing
`<ul> <ol> <li>`	lists	`border`, `border-radius`	borders
`<header> <nav> <main>`	landmarks	`display: flex`	1-D layout
`<article> <section> <aside> <footer>`	landmarks	`display: grid`	2-D layout
`<table> <tr> <th> <td>`	data tables	`gap`	space between items
`<form> <label> <input>`	forms	`@media (min-width: …)`	breakpoints
`<button type="button">`	actions	`--name`, `var(--name)`	tokens
`<select> <textarea>`	form controls	`:hover`, `:focus-visible`	states
`<script src defer>`	JavaScript	`transition`	animation

JavaScript	Use
`const`, `let`	declare variables
`if / else`, `===`, `&&`, `||`	decisions
`for`, `for…of`	loops
`arr.map()`, `filter()`, `reduce()`	transform, select, combine
`document.querySelector()`, `querySelectorAll()`	select elements
`el.textContent`, `el.classList.toggle()`	change text and classes
`document.createElement()`, `el.append()`, `el.remove()`	add and remove elements
`el.addEventListener("click", fn)`, `event.preventDefault()`	events
`JSON.parse()`, `JSON.stringify()`	JSON
`await fetch(url)`, `response.ok`, `response.json()`	load data
`localStorage.setItem()`, `getItem()`	save small data

APPENDIX C · GLOSSARY

ឧបសម្ព័ន្ធ C · សទ្ទានុក្រម

English	ខ្មែរ	English	ខ្មែរ
accessibility	ភាពងាយស្រួលប្រើសម្រាប់មនុស្សគ្រប់គ្នា	element	ធាតុ
alternative text	អត្ថបទជំនួស	event	ព្រឹត្តិការណ៍
API	ចំណុចប្រទាក់កម្មវិធី	form	ទម្រង់
array	អារេ	function	អនុគមន៍
attribute	គុណលក្ខណៈ	heading	ចំណងជើង
breakpoint	ចំណុចបត់បែន	hierarchy	ហានានុក្រម
browser	កម្មវិធីរុករក	link	តំណភ្ជាប់
cascade	លំដាប់អាទិភាពនៃច្បាប់ CSS	media query	លក្ខខណ្ឌតាមអេក្រង់
client	ម៉ាស៊ីនភ្ញៀវ	object	វត្ថុ
commit	ការរក្សាទុកកំណែ	promise	ការសន្យា (តម្លៃនាពេលក្រោយ)
component	សមាសធាតុ	property	លក្ខណៈ
contrast ratio	អនុបាតពណ៌ផ្ទុយ	responsive design	ការរចនាឆ្លើយតប
custom property	អថេរ CSS	selector	អ្នកជ្រើសធាតុ
declaration	ការប្រកាស	semantic	មានអត្ថន័យ
design token	តម្លៃរចនាមានឈ្មោះ	server	ម៉ាស៊ីនមេ
DOM	គំរូវត្ថុឯកសារ	specificity	កម្រិតជាក់លាក់នៃ selector
domain name	ឈ្មោះដែន	state	ស្ថានភាព
DevTools	ឧបករណ៍អ្នកអភិវឌ្ឍ	tag	ស្លាក
Flexbox	ប្លង់បត់បែនមួយវិមាត្រ	usability	ភាពងាយស្រួលប្រើ
Grid	ប្លង់ក្រឡាពីរវិមាត្រ	validation	ការផ្ទៀងផ្ទាត់
HTTP	ពិធីការផ្ទេរអត្ថបទ	viewport	ផ្ទៃបង្ហាញ
JSON	ទម្រង់ទិន្នន័យអត្ថបទ	wireframe	ប្លង់ព្រាង

REFERENCES

ឯកសារយោង

[1] MDN Web Docs. *How the web works*. Mozilla. https://developer.mozilla.org/en-US/docs/Learn/Getting_started_with_the_web/How_the_Web_works

[2] Fielding, R., Nottingham, M., & Reschke, J. (Eds.). (2022). *HTTP Semantics* (RFC 9110). IETF. https://www.rfc-editor.org/rfc/rfc9110

[3] WHATWG. *HTML Living Standard*. https://html.spec.whatwg.org

[4] MDN Web Docs. *HTML, CSS and JavaScript references*. Mozilla. https://developer.mozilla.org

[5] Anthropic. *Prompting best practices*. Claude Developer Platform documentation. https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/claude-prompting-best-practices

[6] W3C. (2023). *Web Content Accessibility Guidelines (WCAG) 2.2*. W3C Recommendation, 5 October 2023. https://www.w3.org/TR/WCAG22/

[7] Duckett, J. (2011). *HTML and CSS: Design and Build Websites*. Wiley.

[8] Krug, S. (2014). *Don't Make Me Think, Revisited: A Common Sense Approach to Web Usability* (3rd ed.). New Riders.

[9] Lupton, E. (2010). *Thinking with Type* (2nd ed.). Princeton Architectural Press.

[10] Nielsen, J. (1994, updated 2024). *10 Usability Heuristics for User Interface Design*. Nielsen Norman Group. https://www.nngroup.com/articles/ten-usability-heuristics/

[11] W3C WAI. *Understanding Success Criterion 1.4.3: Contrast (Minimum)*. https://www.w3.org/WAI/WCAG22/Understanding/contrast-minimum

[12] W3C WAI. (2023). *What's New in WCAG 2.2*. https://www.w3.org/WAI/standards-guidelines/wcag/new-in-22/

[13] W3C CSS Working Group. *CSS Snapshot* (current). https://www.w3.org/TR/CSS/

[14] Marcotte, E. (2010). Responsive web design. *A List Apart*, 306. https://alistapart.com/article/responsive-web-design/

[15] web.dev. (2025). *A definition update for Baseline*. Google. https://web.dev/blog/baseline-definition-update

[16] Frost, B. (2016). *Atomic Design*. Self-published. https://atomicdesign.bradfrost.com

[17] Ecma International. *ECMAScript Language Specification* (ECMA-262). https://tc39.es/ecma262/

[18] Haverbeke, M. (2024). *Eloquent JavaScript* (4th ed.). No Starch Press. https://eloquentjavascript.net

[19] Bray, T. (Ed.). (2017). *The JavaScript Object Notation (JSON) Data Interchange Format* (RFC 8259). IETF.

[20] WHATWG. *Fetch Standard*. https://fetch.spec.whatwg.org

[21] Chrome for Developers. *Lighthouse overview*. Google. https://developer.chrome.com/docs/lighthouse/

[22] GitHub Docs. *Quickstart for GitHub Pages*. https://docs.github.com/en/pages/quickstart

Online sources were checked in October 2026. Web standards are "living": always check MDN Web Docs for the current status of a feature.